

Ontology Modeling

Version 1.4.2-SNAPSHOT, 2026-09-21T22:14:39Z

Table of Contents

1. Ontologies in Quantum: Modeling Relationships That Are Resilient and Fast	1
1.1. What is an Ontology?	1
1.2. Ontology vs. Object Model	1
1.3. Why prefer Ontology-driven relationships over @Reference/EntityReference	2
1.4. How Quantum supports Ontologies	3
1.5. Modeling guidance: from object fields to predicates	4
1.6. Querying with ontology edges vs direct references	4
1.7. Migration: from @Reference to ontology edges	4
1.8. Performance and operational notes	5
1.9. How this integrates with Functional Areas/Domains	5
1.10. Summary	5
2. Concrete example: Sales Orders, Shipments, and evolving to Fulfillment>Returns	6
3. Visualizing ontology edges and provenance	11
3.1. Example graph: Orders, Customers, Organizations	11
3.2. Example graph: Shipments and Regions	11
3.3. What is provenance and how to read it	11
3.4. Querying Ontologies	12
3.4.1. BIAPI Query Language Functions: hasEdge, hasOutgoingEdge, and hasIncomingEdge ..	13
3.4.2. Programmatic Querying with OntologyEdgeRepo	13
3.5. Dynamic Edge Properties, Edge Qualification, and Facet Segmentation	14
3.5.1. 1. Dynamic Edge Properties & Compound Indexes	14
3.5.2. 2. Store-Free Coordinate SPI (<code>DataDomainResolver</code>)	14
3.5.3. 3. Multi-Tenant Policies and Dynamic Facets	15
3.5.4. 4. Defaults, Precedence, and Security Invariants	15
3.5.5. 5. JavaScript Policy Runtime Support	16
3.6. Graph Edge Metadata, Temporal Validity, and Custom Coordinates Variable Resolution	16
3.6.1. 1. Custom Coordinates & Dynamic Variable Resolution (<code>\${facet.}</code> , <code>\${coord.}</code>)	17
3.6.2. 2. Temporal Edge Validity (<code>validFrom</code> / <code>validTo</code>) & the <code>@asOf(ts)</code> Macro	18
3.6.3. 3. Security Classification & Compartments (<code>securityLabel</code> , <code>compartments</code>)	19
3.6.4. 4. AI & Probabilistic Edge Governance (<code>confidence</code> , <code>assertionMethod</code> , <code>@minConfidence(val)</code>)	20
3.6.5. 5. Purpose-Based Access Control (PBAC) & Consent Tracking (<code>allowedPurposes</code> , <code>consentId</code>)	20
3.6.6. 6. Cryptographic Attestation & Separation of Duties (<code>EdgeAttestation</code> , <code>mutuallyExclusiveWith</code>)	21
3.6.7. 7. Summary Matrix of Edge Metadata Dimensions & Defaults	22
4. Integrating Ontology with Morphia, Permissions, and Multi-tenancy	24
4.1. Big picture: where ontology fits	24

4.2. Rule language: add hasEdge()	24
4.3. Passing tenantId correctly	25
4.4. Morphia repository integration patterns	25
4.5. Where ontology materialization happens	26
4.6. Custom Edge Providers: OntologyEdgeProvider and HierarchyListEdgeProvider	26
4.6.1. OntologyEdgeProvider: The Core SPI	26
4.6.2. HierarchyListEdgeProvider: Handling Complex Hierarchies	27
4.6.3. When to use Property Chains vs ComputedEdgeProvider	28
4.6.4. Extending materialization with ComputedEdgeProvider (Advanced)	31
4.6.5. Extending materialization with OntologyEdgeProvider (SPI)	33
4.6.6. Configuration	35
4.6.7. Example: annotate and save — edges are materialized automatically	35
4.7. Security and multi-tenant considerations	35
4.8. Provisional vs. admitted vocabulary tier	36
4.9. Developer workflow and DX checklist	37
4.10. Cookbook: end-to-end example with Orders + Org	37
4.11. Migration notes for teams using @Reference	37
5. Primer: First-time guide to core relationship semantics	38
6. New ontology features in this release	40
6.1. Feature overview	40
6.2. Business problems solved	40
6.3. How to code it with current APIs	41
6.3.1. 1) Define ontology (programmatic TBox)	41
6.3.2. 2) Materialize inferences when data changes	43
6.3.3. 3) Query with ListQueryRewriter (Quarkus/Morphia)	43
6.3.4. Functional properties at write time	44
6.4. Backward compatibility and migration	44
6.5. Troubleshooting and tips	44
7. Model-first ontology with annotations and MorphiaOntologyLoader	46
7.1. Why model-first?	46
7.2. Annotations overview	46
7.2.1. Relationship semantics on @OntologyProperty	47
7.2.2. Cascading policies for relationships	48
7.2.3. Business benefits of ontology relationships and cascade	49
7.3. Example: Orders placed in an Organization (e-commerce)	49
7.4. How the registry is produced (Quarkus CDI)	50
7.5. Defining an ontology with YAML (YamlOntologyLoader)	51
7.6. Materializing inferences from annotated models	53
7.7. Querying using ListQueryRewriter (unchanged)	54
7.8. Business case recipes with annotations	54
7.9. Troubleshooting	54

7.10. Migration strategy	55
7.11. Modeling guidance: annotate existing id/reference getters (keep API and DB clean)	55
8. Exploring the ontology via REST and JointJS	57
8.1. Base info	57
8.2. TBox (schema) endpoints	57
8.3. ABox (instances) graph endpoint	58
8.4. Rendering with JointJS (quick snippet)	58
8.5. When to use these endpoints	59
9. Developer how-to: YAML properties, chains, write-time materialization, reindex, and provenance	60
9.1. Declare properties and chains in YAML	60
9.2. Write-time materialization: what runs and how to toggle	61
9.3. Reindex after YAML changes	62
9.4. Edge provenance format and deletions	63

Chapter 1. Ontologies in Quantum: Modeling Relationships That Are Resilient and Fast



Looking for the short implementation plan? See PROPOSAL.md at the repository root for a concise module-by-module checklist.

This section explains what an ontology is, how it differs from a traditional object model, and how the Quantum Ontology modules make it practical to apply ontology ideas to your domain models and queries. It also contrasts ontology-driven relationships with direct object references (for example, using `@Reference` or `EntityReference`).

1.1. What is an Ontology?

In software terms, an ontology is a formal, explicit specification of concepts and their relationships.

- **Concepts (Classes):** Named categories/types in your domain. Concepts can form taxonomies (is-a hierarchies), be declared disjoint, or be equivalent.
- **Relationships (Properties):** Named relationships between entities. Properties can have a domain (applies to X) and a range (points to Y). They may be inverse or transitive.
- **Axioms (Rules):** Constraints and entailment rules, including property chains such as: if $(A \text{ --p--} B)$ and $(B \text{ --q--} C)$ then we infer $(A \text{ --r--} C)$.
- **Inference:** The process of deriving new facts (types, labels, edges) that were not explicitly stored but follow from axioms and known facts.

An ontology is not the data; it is the schema plus logic that gives your data additional meaning and enables consistent, automated inferences.

1.2. Ontology vs. Object Model

A conventional object model focuses on concrete classes, fields, and direct references between objects at implementation time. An ontology focuses on semantic types and relationships, with explicit rules that can derive new knowledge independent of how objects are instantiated.

Key differences:

Purpose

- **Object model:** Encapsulate data and behavior for application code generation and persistence.
- **Ontology:** Encode shared meaning, constraints, and inference rules that remain stable as implementation details change.

Relationship handling

- **Object model:** Typically uses direct references or foreign keys; traversals are hard-coded and fragile to change.

- **Ontology:** Uses named predicates (properties) and can infer additional relationships by rules (property chains, inverses, transitivity).

Polymorphism and evolution

- **Object model:** Polymorphism requires class inheritance in code; cross-cutting categories are awkward to add later.
- **Ontology:** Entities can have multiple types/labels at once. New concepts and properties can be introduced without breaking existing data.

Querying

- **Object model:** Queries couple to concrete classes and field paths; changes force query rewrites.
- **Ontology:** Queries target semantic relationships; reasoners can materialize edges that queries reuse, decoupling queries from implementation details.

1.3. Why prefer Ontology-driven relationships over @Reference/EntityReference

Direct references (@Reference or custom EntityReference) are simple to start but become restrictive as domains grow:

- **Tight coupling:** Code and queries couple to concrete field paths (customer.primaryAddress.id), making refactors risky.
- **Limited expressivity:** Hard to encode and reuse higher-order relationships (e.g., "partners of my supplier's parent org").
- **Poor polymorphism:** References point to one collection/type; accommodating multiple target types requires extra code.
- **Performance pitfalls:** Deep traversals cause extra queries, N+1 selects, or complex \$lookup joins.

Ontology-driven edges address these issues:

- **Decoupling via predicates:** Use named predicates (e.g., hasAddress, memberOf, supplies) that remain stable while internal object fields change.
- **Inference for reachability:** Property chains can materialize implied links ($A \xrightarrow{p} B \ \& \ B \xrightarrow{q} C \Rightarrow A \xrightarrow{r} C$), avoiding runtime multi-hop traversals.
- **Polymorphism-first:** A predicate can connect heterogeneous types; type inferences (domain/range) remain consistent.
- **Query performance:** Pre-materialized edges allow single-hop, index-friendly queries (in or eq filters) instead of ad-hoc multi-collection traversals.
- **Resilience to change:** You can add or modify rules without rewriting data structures or touching referencing fields across models.

1.4. How Quantum supports Ontologies

Quantum provides three cooperating modules that make ontology modeling practical and fast:

- quantum-ontology-core (package `com.e2eq.ontology.core`)
- `OntologyRegistry`: Holds the TBox (terminology) of your ontology.
- `ClassDef`: Concept names and relationships (parents, disjointWith, sameAs).
- `PropertyDef`: Property names with optional domain, range, inverse flags, and transitivity.
- `PropertyChainDef`: Rules that define multi-hop implications (chains → implied property).
- `TBox`: Container for classes, properties, and property chains.
- `Reasoner` interface and `ForwardChainingReasoner`: Given an entity snapshot and the registry, computes inferences:
- New types/labels to assert on entities.
- New edges to add (implied by property chains, inverses, or other rules).
- quantum-ontology-mongo (package `com.e2eq.ontology.mongo`)
- `EdgeDao`: A thin DAO around an edges collection in Mongo. Each edge contains `tenantId`, `src`, predicate `p`, `dst`, `inferred` flag, `provenance`, and `timestamp`.
- `OntologyMaterializer`: Runs the Reasoner for an entity snapshot and upserts the inferred edges, so queries can be rewritten to simple in/in eq filters.
- quantum-ontology-policy-bridge (package `com.e2eq.ontology.policy`)
- `ListQueryRewriter`: Takes a base query and rewrites it using the `EdgeDao` to filter by the set of source entity ids that have a specific predicate to a given destination.
- This integrates ontology edges with `RuleContext` or policy decisions: policy asks for entities related by a predicate; the rewriter converts that into an efficient Mongo query.
- quantum-ontology-policy-bridge (package `com.e2eq.ontology.policy.rest`)
- `OntologyAwareResource`: Abstract REST base class that extends `BaseResource` to add ontology list (`GET /{path}/ontology`) and detail context (`GET /{path}/id/{id}/ontology`) endpoints. Supports ontology constraints and optional aggregation/expand for combined attributes spanning collections. See [OntologyAwareResource in REST CRUD](#) for usage.

These modules let you define your ontology (core), materialize derived relations (mongo), and leverage them in access and list queries (policy bridge).



In Quarkus, all ontology components are CDI-managed. Inject `EdgeDao` and services via `@Inject`; indexes are ensured automatically at startup by `OntologyMongoProducers`. Configure collection/database with properties `ontology.mongo.database` and `ontology.mongo.collection.edges`.

1.5. Modeling guidance: from object fields to predicates

- Name relationships explicitly
- Define clear predicate names (hasAddress, memberOf, supplies, owns, assignedTo). Avoid encoding relationship semantics in field names only.
- Keep object model minimal and flexible
- Store lightweight identifiers (ids) as needed, but avoid deeply nested reference graphs that encode traversals in code.
- Model polymorphic relationships
- Prefer predicates that naturally connect multiple possible types (e.g., assignedTo can target User, Team, Bot) and rely on ontology type assertions to constrain where needed.
- Use property chains for common paths
- If business logic often traverses $A \rightarrow B \rightarrow C$, define a chain $p \sqsubseteq q \Rightarrow r$ and materialize r for faster queries and simpler policies.
- Capture inverses and transitivity
- For natural inverses (parentOf $\sqsubseteq$ childOf) or transitive relations (partOf, locatedIn), define them in the ontology so edges and queries stay consistent.
- Keep provenance
- Record why an edge exists (prov.rule, prov.inputs) so you can recompute, audit, or retract when inputs change.

1.6. Querying with ontology edges vs direct references

- Direct reference example (fragile/slow)
- Query: "Find Orders whose buyer belongs to Org X or its parents."
- With @Reference: requires joining $\text{Order} \rightarrow \text{User} \rightarrow \text{Org}$ and recursing org.parent; costly and tightly coupled to fields.
- Ontology edge example (resilient/fast)
- Define predicates: placedBy(order, user), memberOf(user, org), ancestorOf(org, org). Define chain placedBy $\sqsubseteq$ memberOf $\Rightarrow$ placedInOrg.
- Materialize edges: (order --placedInOrg--> org). Also make ancestorOf transitive.
- Query becomes: where order._id in EdgeDao.srcIdsByDst(tenantId, "placedInOrg", orgX).
- With transitivity, you can precompute ancestor closure or add a chain placedInOrg $\sqsubseteq$ ancestorOf $\Rightarrow$ placedInOrg to include parents automatically.

1.7. Migration: from @Reference to ontology edges

- Start by introducing predicates alongside existing references; do not remove references

immediately.

- Materialize edges for hot read paths; keep provenance so you can reconstruct.
- Gradually update queries (list screens, policy filters) to use `ListQueryRewriter` with `EdgeDao` instead of deep traversals or `$lookup`.
- Once stable, you can simplify models by removing rigid reference fields where unnecessary and rely on edges for read-side composition.

1.8. Performance and operational notes

- Indexing: Create compound indexes on edges: `(tenantId, p, dst)` and `(tenantId, src, p)` to support both reverse and forward lookups.
- Write amplification vs read wins: Materialization adds write work, but dramatically improves read latency and simplifies queries.
- Consistency: Re-materialize edges on relevant entity changes (source, destination, or intermediate) using `OntologyMaterializer`.
- Multi-tenancy: Keep `tenantId` in the edge key and filters; the provided `EdgeDao` methods include tenant scoping.

1.9. How this integrates with Functional Areas/Domains

- Functional domains often map to concept clusters in the ontology. Use `@FunctionalMapping` to aid discovery and apply policies per area/domain.
- Policies can refer to relationships semantically ("`hasEdge placedInOrg OrgX`") and rely on the policy bridge to turn this into efficient data filters.

1.10. Summary

- Ontology-powered relationships provide a stable, semantic layer over your object model.
- The Quantum Ontology modules let you define, infer, and query these relationships efficiently on MongoDB.
- Compared with direct `@Reference/EntityReference`, ontology edges are more expressive, resilient to change, and typically faster for complex list/policy queries once materialized.

Chapter 2. Concrete example: Sales Orders, Shipments, and evolving to Fulfillment/Returns

This example shows how to use an ontology to model relationships around Orders, Customers, and Shipments, and how the model can evolve to include Fulfillment and Returns without breaking existing queries. We will:

- Define core concepts and predicates.
- Add property chains that materialize implied relationships for fast queries.
- Show how queries are rewritten using edges instead of deep object traversals.
- Evolve the model to support Fulfillment and Returns with minimal changes.

Core concepts (classes)

- Order, Customer, Organization, Shipment, Address, Region
- Later evolution: FulfillmentTask, FulfillmentUnit, ReturnRequest, ReturnItem, RMA

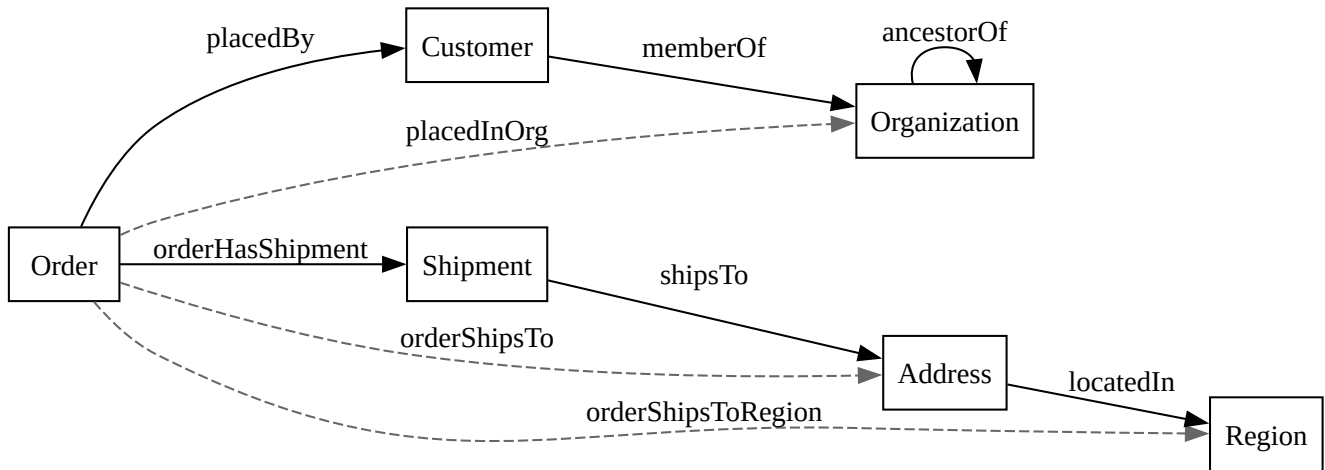
Key predicates (relationships)

- `placedBy(order, customer)`: who placed the order
- `memberOf(customer, org)`: a customer belongs to an organization (or account)
- `orderHasShipment(order, shipment)`: outbound shipment for the order
- `shipsTo(shipment, address)`: shipment destination
- `locatedIn(address, region)`: address is located in a Region
- `ancestorOf(org, org)`: organizational ancestry (transitive)

Property chains (implied relationships)

- `placedBy` $\square$ `memberOf` $\Rightarrow$ `placedInOrg`
- If `(order --placedBy-> customer)` and `(customer --memberOf-> org)`, then infer `(order --placedInOrg-> org)`
- `orderHasShipment` $\square$ `shipsTo` $\Rightarrow$ `orderShipsTo`
- If `(order --orderHasShipment-> shipment)` and `(shipment --shipsTo-> address)`, infer `(order --orderShipsTo-> address)`
- `orderShipsTo` $\square$ `locatedIn` $\Rightarrow$ `orderShipsToRegion`
- If `(order --orderShipsTo-> address)` and `(address --locatedIn-> region)`, infer `(order --orderShipsToRegion-> region)`
- `placedInOrg` $\square$ `ancestorOf` $\Rightarrow$ `placedInOrg`
- Makes `placedInOrg` resilient to org hierarchy changes (`ancestorOf` is transitive). This is a common “closure” trick: re-assert the same predicate via chain to absorb hierarchy.

Diagram of the core classes and predicates for this example:



A minimal Java-style snippet to define this TBox

```

import java.util.*;
import com.e2eq.ontology.core.OntologyRegistry;
import com.e2eq.ontology.core.OntologyRegistry.*;

Map<String, ClassDef> classes = Map.of(
    "Order", new ClassDef("Order", Set.of(), Set.of(), Set.of()),
    "Customer", new ClassDef("Customer", Set.of(), Set.of(), Set.of()),
    "Organization", new ClassDef("Organization", Set.of(), Set.of(), Set.of()),
    "Shipment", new ClassDef("Shipment", Set.of(), Set.of(), Set.of()),
    "Address", new ClassDef("Address", Set.of(), Set.of(), Set.of()),
    "Region", new ClassDef("Region", Set.of(), Set.of(), Set.of())
);

Map<String, PropertyDef> props = Map.of(
    "placedBy", new PropertyDef("placedBy", Optional.of("Order"), Optional.of("Customer"), false, Optional.empty(), false),
    "memberOf", new PropertyDef("memberOf", Optional.of("Customer"), Optional.of("Organization"), false, Optional.empty(), false),
    "orderHasShipment", new PropertyDef("orderHasShipment", Optional.of("Order"), Optional.of("Shipment"), false, Optional.empty(), false),
    "shipsTo", new PropertyDef("shipsTo", Optional.of("Shipment"), Optional.of("Address"), false, Optional.empty(), false),
    "locatedIn", new PropertyDef("locatedIn", Optional.of("Address"), Optional.of("Region"), false, Optional.empty(), false),
    "ancestorOf", new PropertyDef("ancestorOf", Optional.of("Organization"), Optional.of("Organization"), false, Optional.empty(), true), // transitive
    // implied predicates (no domain/range required, but you may add them for validation)
    "placedInOrg", new PropertyDef("placedInOrg", Optional.of("Order"), Optional.of("Organization"), false, Optional.empty(), false),
    "orderShipsTo", new PropertyDef("orderShipsTo", Optional.of("Order"), Optional.of("Address"), false, Optional.empty(), false),
    "orderShipsToRegion", new PropertyDef("orderShipsToRegion", Optional.of("Order"),

```

```
Optional.of("Region"), false, Optional.empty(), false)
);

List<PropertyChainDef> chains = List.of(
    new PropertyChainDef(List.of("placedBy", "memberOf", "placedInOrg"),
    new PropertyChainDef(List.of("orderHasShipment", "shipsTo", "orderShipsTo"),
    new PropertyChainDef(List.of("orderShipsTo", "locatedIn", "orderShipsToRegion"),
    new PropertyChainDef(List.of("placedInOrg", "ancestorOf", "placedInOrg")
);

OntologyRegistry.TBox tbox = new OntologyRegistry.TBox(classes, props, chains);
OntologyRegistry registry = OntologyRegistry.inMemory(tbox);
```

Materializing edges for an Order

- Explicit facts for order O1:
- O1 placedBy C9
- C9 memberOf OrgA
- O1 orderHasShipment S17
- S17 shipsTo Addr42
- Addr42 locatedIn RegionWest
- OrgA ancestorOf OrgParent
- Inferred edges after running the reasoner for O1's snapshot:
- O1 placedInOrg OrgA
- O1 placedInOrg OrgParent (via closure with ancestorOf)
- O1 orderShipsTo Addr42
- O1 orderShipsToRegion RegionWest

How queries become simple and fast

- List Orders for Organization OrgParent (including children):
- Instead of joining Order → Customer → Org and recursing org.parent, run a single filter using materialized edges.

```
import com.mongodb.client.model.Filters;
import org.bson.conversions.Bson;
import com.e2eq.ontology.policy.ListQueryRewriter;

Bson base = Filters.eq("status", "OPEN");
Bson rewritten = rewriter.rewriteForHasEdge(base, tenantId, "placedInOrg", "OrgParent");
// Use rewritten in your Mongo find
```

- List Orders shipping to RegionWest:

```
Bson rewritten2 = rewriter.rewriteForHasEdge(Filters.empty(), tenantId,
"orderShipsToRegion", "RegionWest");
```

Why this is resilient

- If tomorrow Customer becomes AccountContact and the organization model gains Divisions and multi-parent org graphs, you only adjust predicates and chains.
- Queries that rely on placedInOrg or orderShipsToRegion remain unchanged and fast, because edges are re-materialized by OntologyMaterializer.

Evolving the model: add Fulfillment

New concepts

- FulfillmentTask: a unit of work to pick/pack/ship order lines
- FulfillmentUnit: a logical grouping (e.g., wave, tote, parcel)

New predicates

- fulfills(task, order)
- realizedBy(order, fulfillmentUnit)
- taskProduces(task, shipment)

New chains (implied)

- fulfills $\Rightarrow$ derived edge from task to order; combine with taskProduces to connect order to shipment without touching Order fields:
- fulfills $\square$ taskProduces $\Rightarrow$ orderHasShipment
- realizedBy $\square$ orderHasShipment $\Rightarrow$ fulfilledByUnit
- If (order --realizedBy- $\rightarrow$ fu) and (order --orderHasShipment- $\rightarrow$ s) $\Rightarrow$ (fu --fulfillsShipment- $\rightarrow$ s) or simply (order --fulfilledByUnit- $\rightarrow$ fu)

These chains let you introduce warehouse concepts without changing how UI filters orders by organization or ship-to region. Existing queries still operate via placedInOrg and orderShipsToRegion.

Evolving further: add Returns

New concepts

- ReturnRequest, ReturnItem, RMA

New predicates

- hasReturn(order, returnRequest)
- returnFor(returnItem, order)

- `returnRma(returnRequest, rma)`

New chains (implied)

- `hasReturn` $\Rightarrow$ `openReturnOnOrg` via `placedInOrg`:
- `hasReturn` $\square$ `placedInOrg` $\Rightarrow$ `returnPlacedInOrg`
- `returnFor` $\square$ `orderShipsToRegion` $\Rightarrow$ `returnShipsToRegion`

Example queries with new capabilities

- List Orders with open returns in OrgParent:

```
Bson r = rewriter.rewriteForHasEdge(Filters.empty(), tenantId, "returnPlacedInOrg",
"OrgParent");
```

- List Returns associated to Orders shipping to RegionWest:

```
Bson r2 = rewriter.rewriteForHasEdge(Filters.empty(), tenantId, "returnShipsToRegion",
"RegionWest");
```

Comparison with direct references (@Reference/EntityReference)

- With direct references you would encode fields like `Order.customer`, `Order.shipments`, `Shipment.address`, `Address.region` and then implement multi-hop traversals in code or \$lookup pipelines, rewriting them whenever you add Fulfillment or Returns.
- With ontology edges, you keep predicates stable and add property chains. Existing list and policy queries keep working and typically become faster due to single-hop filters on an indexed edges collection.

Operational tips for this scenario

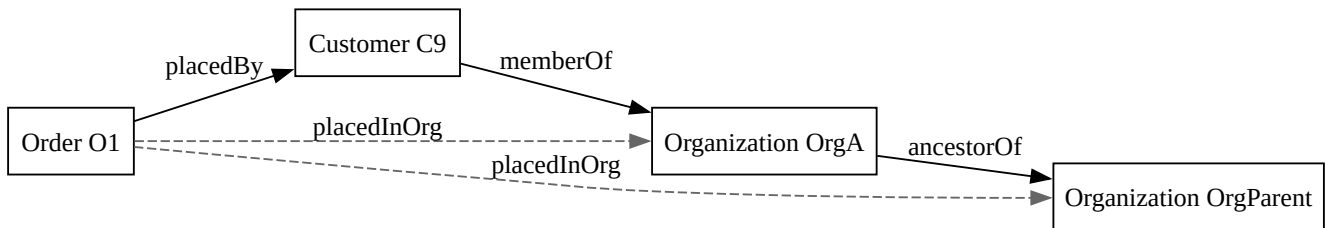
- Ensure `EdgeDao` has indexes on `(tenantId, p, dst)` and `(tenantId, src, p)`.
- Use `OntologyMaterializer` when `Order`, `Shipment`, `Customer`, `Address`, or org hierarchy changes to keep edges fresh.
- Keep provenance in `edge.prov` (rule, inputs) so you can recompute or retract edges when source data changes.

Chapter 3. Visualizing ontology edges and provenance

This section uses simple diagrams to illustrate how explicit facts (edges) combine with ontology rules (property chains, inverses, transitivity) to produce inferred edges, and how provenance explains each inference.

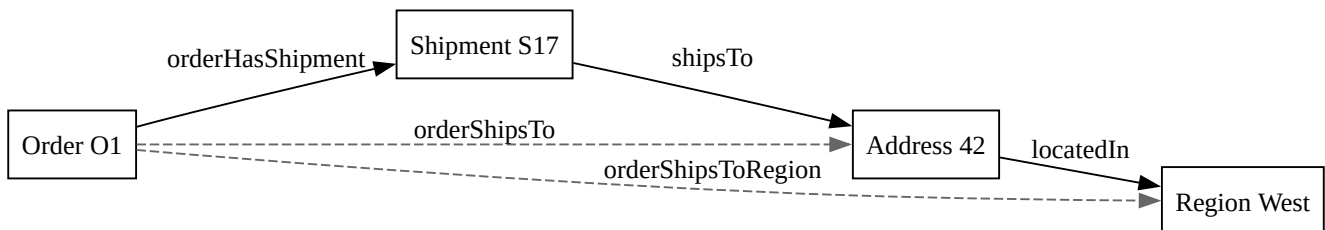
3.1. Example graph: Orders, Customers, Organizations

We start from the running example introduced above. The following diagram shows explicit edges in solid lines and inferred edges as dotted lines.



You can add more nodes and predicates (shipments, addresses, regions) and draw similar diagrams. The important idea is that you query for semantics (placedInOrg), while the reasoner ensures the materialized edges exist according to the rules.

3.2. Example graph: Shipments and Regions



3.3. What is provenance and how to read it

Every inferred edge includes a provenance payload prov that answers “why does this edge exist?”. It captures the rule used and the contributing input edges. With recent enrichments, inputs also include type information when available.

Here is an example edge document (simplified JSON for clarity) for the inference `O1 --placedInOrg--> OrgA` via the chain `placedBy ◻ memberOf ⇒ placedInOrg`:

```
{
  "tenantId": "t1",
  "src": "O1",
  "srcType": "Order",
  "p": "placedInOrg",
  "dst": "OrgA",
```

```

"dstType": "Organization",
"inferred": true,
"prov": {
  "rule": "chain",
  "inputs": {
    "chain": ["placedBy", "memberOf"],
    "inputs": [
      { "src": "O1", "srcType": "Order", "p": "placedBy", "dst": "C9",
"dstType": "Customer" },
      { "src": "C9", "srcType": "Customer", "p": "memberOf", "dst": "OrgA",
"dstType": "Organization" }
    ]
  }
},
"ts": "2025-10-20T10:01:02Z"
}

```

Notes: - rule indicates which inference rule created the edge (chain, inverse, symmetric, transitive, subPropertyOf). - inputs lists the contributing explicit edges (or minimal hops) that justified the inference. When types are known from domain/range or snapshots, srcType and dstType are included. - This provenance enables audit, troubleshooting, and selective recomputation when inputs change.

For a transitive example, an O1 --placedInOrg--> OrgParent edge could have:

```

{
  "prov": {
    "rule": "transitive",
    "inputs": {
      "prop": "placedInOrg",
      "inputs": [ { "src": "O1", "p": "placedInOrg", "dst": "OrgA", "srcType": "Order",
"dstType": "Organization" } ]
    }
  }
}

```



Provenance is for explanation and maintenance. The runtime edges you query on remain simple (src, p, dst) with optional type fields for validation and indexing.

3.4. Querying Ontologies

Quantum provides several ways to leverage ontology relationships in your queries and business logic.

3.4.1. BIAPI Query Language Functions: `hasEdge`, `hasOutgoingEdge`, and `hasIncomingEdge`

The query language (BIAPIQuery) includes built-in functions to filter entities based on their ontology relationships. These functions support both standard relationship existence checks and qualified property filtering:

- `hasEdge(predicate, dst [, { edgeFilter }])`: Filters for entities (sources) that have an edge pointing TO a specific destination, optionally satisfying edge-level property constraints.
- `hasOutgoingEdge(predicate, dst [, { edgeFilter }])`: Synonym for `hasEdge`.
- `hasIncomingEdge(predicate, src [, { edgeFilter }])`: Filters for entities (targets) that have an edge coming FROM a specific source, optionally satisfying edge-level property constraints.

Example BIAPI queries:

```
# Find orders placed in OrgA
hasEdge(placedInOrg, "OrgA")

# Find orders placed in OrgA with high priority and active status
hasEdge("placedInOrg", "OrgA", { priority:"HIGH" && active:true })

# Find locations accessible by a specific associate with tier 1 or 2 access
hasIncomingEdge(canAccessLocation, "associate-123", { tier:^[#1, #2] })

# Combine with root entity filters
hasEdge(placedInOrg, "OrgA", { department:"FINANCE" }) && status:PENDING
```

3.4.2. Programmatic Querying with `OntologyEdgeRepo`

For programmatic access in Java services, use `OntologyEdgeRepo` directly:

```
@Inject
OntologyEdgeRepo edgeRepo;

// 1. Basic destination lookup (unfiltered)
Set<String> targetIds = edgeRepo.dstIdsBySrc(tenantId, "Associate", associateId,
"canAccessLocation");

// 2. Qualified edge filtering with MongoDB property filters (pushed into compound
index scan)
Set<String> approvedSources = edgeRepo.srcIdsByDst(
    tenantId,
    "placedInOrg",
    "OrgA",
    Filters.eq("props.priority", "HIGH"),
    Filters.eq("props.active", true)
);
```

```
// 3. Upserting edges with dynamic property maps
Map<String, Object> props = Map.of(
    "priority", "HIGH",
    "active", true,
    "effectiveDate", Instant.now().toString()
);
edgeRepo.upsert(tenantId, "Order", orderId, "placedInOrg", "Org", orgId, props);
```

3.5. Dynamic Edge Properties, Edge Qualification, and Facet Segmentation

As multi-tenant domains evolve, access boundaries often depend on multidimensional runtime facets (such as organization units, geographic regions, partner tiers, and classification levels) rather than static relational foreign keys. Quantum decouples multi-tenant data segmentation semantics from MongoDB physical persistence mechanics using **Dynamic Edge Properties**, **Qualified Edge Traversal**, and the **Store-Free Coordinate SPI**.

3.5.1. 1. Dynamic Edge Properties & Compound Indexes

Every `OntologyEdge` and transport-neutral `EdgeRecord` includes a flexible `props: Map<String, Object>` payload:

- **Entity Model (`OntologyEdge`):** Holds arbitrary key-value metadata characterizing the relationship (e.g. `role`, `validFrom`, `validTo`, `serviceLevel`, `department`).
- **Compound Index Scans:** The underlying MongoDB `edges` collection is backed by compound read-optimizing indexes:
 - `idx_domain_p_dst_props`: `{"dataDomain": 1, "p": 1, "dst": 1, "props": 1}`—optimizes `hasEdge` / `srcIdsByDst` lookups.
 - `idx_domain_src_p_props`: `{"dataDomain": 1, "src": 1, "p": 1, "props": 1}`—optimizes `hasIncomingEdge` / `dstIdsBySrc` lookups.
- **Dynamic Property Scoping:** When querying edges via `{ edgeFilter }`, all non-root field references are automatically prefixed with `props.` so queries can naturally write `{ role:"ADMIN" }` instead of `{ props.role:"ADMIN" }`.

3.5.2. 2. Store-Free Coordinate SPI (`DataDomainResolver`)

In multi-tenant SaaS environments, different tenants may require different coordinate resolution schemes (e.g. resolving tenant, account, department, or geographic facet coordinates). Quantum defines a storage-agnostic SPI in `quantum-models`:

```
package com.e2eq.framework.model.security;

public interface DataDomainResolver {
    // Resolve coordinates when inserting or updating entities
    DataDomainResolution resolveForCreate(Class<?> modelClass, SourceAttributes
```

```

source);

    // Resolve query constraints for root list/find operations
    DataDomainResolution resolveForQuery(Class<?> modelClass, String action,
PrincipalContext context);

    // Resolve query constraints when hopping across relationships in aggregation
pipelines
    DataDomainResolution resolveForHop(Class<?> targetClass, String viaPath,
PrincipalContext context);
}

```

- **SourceAttributes:** A store-free, read-only wrapper around model fields, decoupling coordinate calculation from Morphia, Hibernate, or MongoDB specifics.
- **DataDomainResolution:** A sealed tagged union (`DataDomainResolution.Resolved` vs `DataDomainResolution.Unresolvable`). If coordinate resolution fails or rules deny access, `Unresolvable` causes the query or mutation to **fail closed**.

3.5.3. 3. Multi-Tenant Policies and Dynamic Facets

A single Quantum server instance supports heterogeneous policies across different tenants:

- **Facet Bindings (`DataDomainComponentBinding`):** Carries open, user-definable facet bindings (`Map<String, Binding> facets`) mapping facet names to model field paths, constant values, or contextual variables.
- **Dynamic Policy Entries (`DataDomainPolicyEntry`):** Encapsulates:
 - **filter:** A BIAPI query expression restricting rows (e.g. `status:active && region:${region}`).
 - **facetFilters:** Structured constraints keyed by facet name (e.g. `department → "FINANCE"`).
- **Tenant Independence:**
 - In dedicated realms, each tenant maintains its own `systemDataDomainPolicies` collection.
 - In shared/pooled realms, policies are isolated by `tenantId`.
 - Different tenants can enforce completely different segmentation rules on the same entity classes.

3.5.4. 4. Defaults, Precedence, and Security Invariants

Component	Default Behavior	Security Invariant
Edge Property Constraints	Omitted { <code>edgeFilter</code> } defaults to an unqualified edge existence check.	An edge without a <code>props</code> map matches unqualified queries, but fails closed on property queries.

Component	Default Behavior	Security Invariant
Coordinate Resolution Precedence	1. Explicit tenant policy for (<code>modelClass</code> , <code>action</code>). 2. System-wide / Global policy provider. 3. Fallback to caller's <code>PrincipalContext</code> credentials (<code>tenantId</code> , <code>ownerId</code> , <code>orgRefName</code>).	If none of the above resolve valid coordinates, <code>DataDomainResolver</code> returns <code>Unresolvable</code> .
Missing Security Context	Execution fails closed with HTTP 500 (<code>SecurityContextMissing</code>).	No query or mutation runs without an authenticated principal or explicit administrative rule bypass.
Rule Context Injection	<code>DataDomainPolicy</code> entries automatically bind variables into <code>RuleContext</code> (<code>\${pTenantId}</code> , <code>\${orgRefName}</code> , <code>\${ownerId}</code> , <code>\${policyFilter}</code> , and typed facet variables).	Downstream repository and query gateway queries automatically inherit these variables for filter evaluation.

3.5.5. 5. JavaScript Policy Runtime Support

Policy scripts executed by `RuleScriptExecutor` can test qualified edges directly in JavaScript pre- and post-conditions:

```
// 1. Match edge with exact property values
var hasPrimaryRole = hasEdge("memberOf", principal.orgRefName, { role: "PRIMARY" });

// 2. Match edge using a JavaScript predicate function
var isHighTier = hasEdge("assignedToTeam", "SUPPORT", function(props) {
  return props && props.tier >= 2 && props.active === true;
});
```

The ontology TBox vocabulary validator (`RuleVocabularyValidator`) validates 3-argument `hasEdge` statements at deployment/registration time against the admitted properties defined in your ontology schema.

3.6. Graph Edge Metadata, Temporal Validity, and Custom Coordinates Variable Resolution

While open property maps (`props`) allow storing unstructured key-value attributes on graph relationships, complex enterprise deployments require **first-class governance metadata** directly on ontology edges (`OntologyEdge` and `EdgeRecord`).

To address regulatory compliance, zero-trust security, AI trust boundaries, and dynamic multi-tenant segmentation, Quantum provides structured edge metadata, query language macros (`@asOf`, `@minConfidence`), and dotted custom coordinate resolution (`${facet.}`, `${coord.}`).

3.6.1. 1. Custom Coordinates & Dynamic Variable Resolution (`${facet.}`, `${coord.}`)

Business Scenario: Multi-Tenant Retail Franchise & Partner Distribution Network

A national retail holding company operates diverse franchised store brands (e.g. Supermarkets, Convenience, and Pharmacies) sharing a centralized product catalog, procurement platform, and order routing pipeline.

Franchise operators, regional distribution managers, and store associates access the same central API endpoints. However, access is governed not by static role names, but by multidimensional business coordinates: - The store's assigned **tier** (e.g. `FLAGSHIP`, `REGIONAL`, `EXPRESS`), - The manager's **geographic zone** (e.g. `US_NORTHEAST`, `US_MIDWEST`), and - The partner's **distribution channel** (e.g. `DIRECT`, `WHOLESALE`).

How the Feature Supports the Business Solution

Without dynamic variable resolution, administrators must either maintain thousands of redundant, hardcoded security rules per franchise, or require application developers to write brittle custom filtering logic into service endpoints.

Quantum solves this by supporting **dotted variable identifiers** (`${facet.tier}`, `${coord.salesZone}`) directly within ANTLR grammar queries and policy rules, resolved dynamically by the store-free `DataDomainResolver` SPI:

- Dotted Identifiers in ANTLR:** The query lexer admits `${facet.name}`, `${coord.name}`, `${principal.attr}`, and `${dd.field}` as first-class variable tokens.
- Namespace Precedence:** During variable bundle compilation (`MorphiaUtils.buildVariableBundle`), namespaces resolve in a deterministic hierarchy:
 - * `${facet.<key>}` & `${coord.<key>}`: Evaluated from tenant `DataDomainPolicyEntry.facetFilters` or contextual coordinates.
 - * `${principal.<attr>}`: Extracted from authenticated principal claims (`userId`, `defaultRealm`, `tenantId`, custom attributes).
 - * `${dd.<attr>}`: Extracted from target `DataDomain` coordinates (`tenantId`, `orgRefName`, `accountNum`, `dataSegment`, `ownerId`).
- Bracketless Single-Variable IN Evaluation (`field: ^${var}`):** When a variable evaluates to a Java `Collection` (e.g. `allowedTiers = ["FLAGSHIP", "REGIONAL"]`), the query listener automatically unpacks the collection into a native MongoDB `$in` operator without requiring bracket syntax `[${var}]`.
- Fail-Closed Isolation:** If an unauthenticated caller or unresolvable coordinate is encountered, variable evaluation fails closed, preventing cross-franchise data leakage.

Policy & Query Example

```
# Declarative Security Rule enforcing custom franchise tiers and geographic zone
id: 5012
name: "Franchise Regional Inventory Access"
securityURI: "sec://inventory/read"
effect: ALLOW
andFilterString: >
  hasEdge("allocatedToStore", "${dcOrgRefName}", { active: true }) &&
  tier: ^${facet.allowedTiers} &&
  zone: ${coord.salesZone} &&
  status: AVAILABLE
```

When an authenticated manager with `allowedTiers = ["FLAGSHIP", "REGIONAL"]` and `salesZone = "US_NORTHEAST"` queries inventory, the query automatically compiles to:

```
{
  "tier": { "$in": ["FLAGSHIP", "REGIONAL"] },
  "zone": "US_NORTHEAST",
  "status": "AVAILABLE"
}
```

3.6.2. 2. Temporal Edge Validity (`validFrom` / `validTo`) & the `@asOf(ts)` Macro

Business Scenario: Hospital Emergency Staffing Rotations & Point-in-Time Audits

In a regional hospital network, medical doctors and nursing staff rotate through shifts and departments on a temporary basis (e.g., Dr. Smith is assigned as "Acting Chief of Emergency Medicine" strictly from Monday 08:00 to Friday 17:00).

Furthermore, under HIPAA and Joint Commission regulatory standards, the hospital must undergo historical point-in-time compliance audits: **"Did Dr. Smith have authorization to access Patient X's intensive care psychiatric records on October 14th at 14:30?"**

How the Feature Supports the Business Solution

Traditional authorization systems attempt to handle temporary access using asynchronous batch cron jobs that delete expired relationship records. This introduces severe vulnerabilities: - **Polling Latency:** The user retains unauthorized access between actual expiration and the next cron run. - **Audit Destruction:** Deleting expired edges permanently destroys historical lineage, making historical point-in-time compliance audits impossible.

Quantum introduces first-class temporal validity directly onto `OntologyEdge` and `EdgeRecord`: - **`validFrom`: Date:** Inclusive timestamp when the relationship becomes active (or `null` for beginning of time). - **`validTo`: Date:** Exclusive timestamp when the relationship expires (or `null` for indefinite validity). - **Compound Temporal Index:** Backed by MongoDB index `idx_domain_temporal` (`{"dataDomain.tenantId": 1, "validFrom": 1, "validTo": 1}`). - **The `@asOf(timestamp)` Macro:** The query normalizer transforms `@asOf(ts)` into an optimized range comparison: `validFrom:<=ts || validFrom:null` && `(validTo:>=ts || validTo:null` (or `edges.` prefixed in traversal contexts).

Query & Code Example

```
# 1. Real-time access policy: Edge must be active right now
hasEdge("actingChiefOf", "DEPT-EMERGENCY", { @asOf(2026-09-20T22:00:00Z) &&
status:"ACTIVE" })

# 2. Historical audit query: Was the physician assigned to this patient on October
14th?
@asOf(2026-10-14T14:30:00Z) && hasEdge("assignedPhysician", "PATIENT-8891")
```

In Java domain code:

```
EdgeRecord edge = new EdgeRecord();
edge.setSrc("physician_smith");
edge.setP("actingChiefOf");
edge.setDst("DEPT-EMERGENCY");
edge.setValidFrom(Date.from(Instant.parse("2026-09-21T08:00:00Z")));
edge.setValidTo(Date.from(Instant.parse("2026-09-25T17:00:00Z")));
edgeRepo.bulkUpsertEdgeRecords(realmId, List.of(edge));
```

3.6.3. 3. Security Classification & Compartments (**securityLabel**, **compartments**)

Business Scenario: Aerospace Defense Contracting & Sensitive Mergers & Acquisitions

An aerospace defense conglomerate collaborates with defense ministries (DoD, NATO) and commercial aerospace manufacturers. In classified defense programs, **the existence of a relationship is itself classified**. For example, the association between an engineering contractor and **PROJECT-TITAN** has a security classification of **SECRET** with an **ITAR** (International Traffic in Arms Regulations) handling compartment.

If an uncleared sub-contractor or foreign national queries their project dashboard, the query must not reveal that **PROJECT-TITAN** is linked to their team, nor leak information via query timing or error messages.

How the Feature Supports the Business Solution

Quantum implements Mandatory Access Control (MAC) directly in the graph datastore layer: - **securityLabel: String**: Hierarchical classification level (e.g. **UNCLASSIFIED**, **CONFIDENTIAL**, **SECRET**, **TOP_SECRET**). - **compartments: List<String>**: Need-to-know compartment restriction codes (e.g. **ITAR**, **NOFORN**, **FINANCE**, **LEGAL**). - **Index Scan Pushdown**: Backed by compound index **idx_domain_security** (**{"dataDomain.tenantId": 1, "securityLabel": 1}**). Security constraints are enforced inside the database index scan: **hasEdge("assignedToProject", "PROJECT-TITAN", { securityLabel: "SECRET" && compartments:^ ["ITAR"] })** - **Zero Information Leakage**: If the caller's **PrincipalContext** lacks the required clearance or compartment entitlements, the edge is withheld at the database level. The result is indistinguishable from the relationship not existing.

Policy Example

```
id: 6100
name: "Defense Project Access Guard"
securityURI: "sec://projects/view"
effect: ALLOW
andFilterString: >
  hasEdge("assignedToProject", "PROJECT-TITAN", {
    securityLabel: "SECRET" &&
    compartments:^ ["ITAR"]
  })
```

3.6.4. 4. AI & Probabilistic Edge Governance (**confidence**, **assertionMethod**, **@minConfidence(val)**)

Business Scenario: Automated Anti-Money Laundering (AML) & AI Entity Deduplication

A global investment bank ingests corporate registries, transaction histories, and sanctions lists to uncover financial crime. An AI pipeline using Large Language Models and vector embeddings continuously generates graph relationships (e.g., **"Vendor Corp is 85% likely to be an alias of Sanctioned Oligarch Shell Co"**).

While fraud investigators require visibility into probabilistic links, automated transaction-blocking systems or multi-million-dollar wire approvals cannot legally act on an 85% probabilistic guess without human certification.

How the Feature Supports the Business Solution

Quantum formalizes confidence scoring and assertion provenance into graph relationships: - **confidence: Double**: Normalized certainty score from **0.0** (unverified conjecture) to **1.0** (verified ground truth). - **assertionMethod: String**: Provenance technique (**MANUAL**, **RULE_DERIVED**, **LLM_INFERENCE**, **VECTOR_SIMILARITY**). - **@minConfidence(val) Macro**: Shorthand macro expanding directly to **confidence:>=val** (or **edges.confidence:>=val**). - **Compound Index Scans**: Backed by MongoDB index **idx_domain_confidence** (**{"dataDomain.tenantId": 1, "confidence": 1}**). - **Risk-Tiered Policy Governance**: High-risk financial operations enforce **confidence: 1.0 && assertionMethod: "MANUAL"**, while exploratory investigative views accept **@minConfidence(0.70)**.

Query Example

```
# 1. Mission-Critical Payment Transfer: Strictly require human-verified relationship
hasEdge("beneficialOwnerOf", "ACC-9921", {
  confidence: 1.0 &&
  assertionMethod: "MANUAL"
})

# 2. Financial Crime Intelligence Dashboard: Permit AI-inferred relationships with >=
80% confidence
hasEdge("beneficialOwnerOf", "ACC-9921", {
  @minConfidence(0.80)
})
```

3.6.5. 5. Purpose-Based Access Control (PBAC) & Consent Tracking (**allowedPurposes**, **consentId**)

Business Scenario: Healthcare Clinical Trials & GDPR Data Privacy Compliance

Under European General Data Protection Regulation (GDPR Articles 6 and 9) and HIPAA, patient healthcare data cannot be accessed for arbitrary purposes. Each access request must carry a legitimate, declared business purpose (e.g. **TREATMENT**, **BILLING**, **CLINICAL_RESEARCH**, **DIRECT_MARKETING**).

A patient signs a consent agreement allowing their clinical records to be linked to attending physicians for **TREATMENT** and **BILLING**, but explicitly withholding consent for **CLINICAL_RESEARCH**. The hospital's centralized data platform must enforce this distinction across clinical trial systems and hospital billing software without duplicating data into isolated silos.

How the Feature Supports the Business Solution

Quantum integrates Purpose-Based Access Control (PBAC) into edge qualification: - **allowedPurposes: List<String>**: Whitelist of declared business purposes for which the relationship may be traversed. - **consentId: String**: Cryptographic or regulatory consent contract identifier. - **Dynamic Purpose Evaluation**: The calling principal's active request purpose (`${principal.requestPurpose}`) is matched against the edge's **allowedPurposes** at the datastore query layer: `hasEdge("caresForPatient", "${patientId}", { allowedPurposes: ^${principal.requestPurpose} })` - If an attending physician accesses the patient with `requestPurpose = "TREATMENT"`, the query matches and permits record access. If a pharmaceutical research team queries the same graph with `requestPurpose = "CLINICAL_RESEARCH"`, the relationship filter rejects the query at the index level.

Policy Example

```
id: 7200
name: "Patient Chart Privacy Policy"
securityURI: "sec://medical-records/read"
effect: ALLOW
andFilterString: >
  hasEdge("attendingPhysicianOf", "${patientId}", {
    allowedPurposes: ^${principal.requestPurpose} &&
    consentId: ~
  })
```

3.6.6. 6. Cryptographic Attestation & Separation of Duties (**EdgeAttestation**, **mutuallyExclusiveWith**)

Business Scenario: B2B Cross-Tenant Supply-Chain Trust & SOX Financial Controls

In an international aerospace manufacturing network, Tier-1 aerospace builders outsource component manufacturing to external suppliers across different enterprise tenants. 1. **Supply-Chain Integrity**: Supplier A's ISO-9001 quality certification edge must be signed by an authorized accredited auditor. The aerospace manufacturer's systems must verify this cryptographic signature without needing administrative access to the auditor's database. 2. **Separation of Duties (SoD)**: Under Sarbanes-Oxley (SOX) Section 404, internal financial controls require strict Separation of Duties: an employee who creates a purchase order cannot be the approver of that same purchase order.

How the Feature Supports the Business Solution

Quantum provides cryptographic verification and graph conflict constraints: - **EdgeAttestation**: A tamper-evident cryptographic signature container: * **keyId**: Identity of the signing public key or certificate thumbprint. * **signature**: Digital signature (Ed25519 / ECDSA) computed over the

canonical edge coordinates (`tenantId + src + p + dst`). * `algorithm`: Cryptographic algorithm used (e.g. "Ed25519", "ES256"). * `timestamp`: Precise signature issuance timestamp. - `mutuallyExclusiveWith: List<String>`: Declares incompatible relationship predicates. A subject holding an edge to an entity cannot simultaneously hold any edge listed in `mutuallyExclusiveWith` to that same entity.

Java Domain & Attestation Example

```
// 1. Creating a cryptographically signed supplier certification edge
EdgeAttestation attestation = new EdgeAttestation(
    "did:key:z6MkuTLjPt...#key-1",
    "f83b2...signature_bytes...",
    "Ed25519",
    new Date()
);

EdgeRecord edge = new EdgeRecord();
edge.setSrc("supplier_acme_corp");
edge.setP("certifiedQualitySupplier");
edge.setDst("ISO-9001-STANDARDS");
edge.setAttestation(attestation);
edge.setMutuallyExclusiveWith(List.of("blacklistedSupplier", "suspendedSupplier"));

edgeRepo.bulkUpsertEdgeRecords("aerospace-realm", List.of(edge));
```

3.6.7. 7. Summary Matrix of Edge Metadata Dimensions & Defaults

Dimension	Field Name	Data Type	Default Behavior	Supported Index / Query Optimization
Temporal Validity	<code>validFrom</code> , <code>validTo</code>	Date	<code>null</code> indicates open-ended validity (from the beginning of time / indefinitely).	<code>idx_domain_temporal</code> + <code>@asOf(ts)</code> query macro.
Classification & Compartments	<code>securityLabel</code> , <code>compartments</code>	String, <code>List<String></code>	<code>null</code> indicates standard unclassified data without handling restrictions.	<code>idx_domain_security</code> + list membership <code>:^</code> .
Certainty & Provenance	<code>confidence</code> , <code>assertionMethod</code>	Double, String	<code>null</code> defaults to untracked confidence (1.0 ground truth implied).	<code>idx_domain_confidence</code> + <code>@minConfidence(val)</code> query macro.
Purpose & Consent (PBAC)	<code>allowedPurposes</code> , <code>consentId</code>	<code>List<String></code> , String	<code>null</code> indicates general operational data; queries specifying purpose filter fail closed if omitted.	Compound index scan with list membership <code>:^</code> .

Dimension	Field Name	Data Type	Default Behavior	Supported Index / Query Optimization
Cryptographic Attestation	attestation	EdgeAttestation	null indicates unsigned internal edge.	BSON embedded document persistence with tamper verification.
Separation of Duties	mutuallyExclusiveWith	List<String>	null indicates no conflicting predicate constraints.	Schema integrity validation during reasoner materialization.

Chapter 4. Integrating Ontology with Morphia, Permissions, and Multi-tenancy

This section focuses on integration and developer experience: how ontology edges flow into Morphia-based repositories and the permission rule language, while remaining fully multi-tenant and secure.

4.1. Big picture: where ontology fits

- Write path (materialization):
- Your domain code persists entities with minimal direct references.
- An `OntologyMaterializer` runs when entities change to derive and upsert edges into the edges collection (per tenant).
- Policy path (authorization and list filters):
- The permission rule language evaluates the caller's `SecurityContext/RuleContext` and produces logical filters.
- When a rule asks for a semantic relationship (`hasEdge`), we use `ListQueryRewriter` + `EdgeDao` to translate that into efficient Mongo filters over ids.
- Read path (queries):
- Morphia repos apply the base data-domain filters and the rewritten ontology constraint to queries, producing fast lists without deep joins.

4.2. Rule language: add `hasEdge()`

We introduce a policy function/operator to reference ontology edges directly from rules:

- Signature: `hasEdge(predicate, dstIdOrVar)`
- predicate: String name of the ontology predicate (e.g., "placedInOrg", "orderShipsToRegion").
- dstIdOrVar: Either a concrete id/refName or a variable resolved from `RuleContext` (e.g., `principal.orgRefName`, `request.region`).
- Semantics: The rule grants/filters entities for which an edge (`tenantId`, `src = entity._id`, `p = predicate`, `dst = resolvedDst`) exists.
- Composition: `hasEdge` can be combined with existing rule clauses (and/or/not) and other filters (states, tags, `ownerId`, etc.).

Example rule snippets (illustrative):

- Allow viewing Orders in the caller's org (including ancestors via ontology closure):
- `allow VIEW Order when hasEdge("placedInOrg", principal.orgRefName)`
- Restrict list to Orders shipping to a region chosen in request:
- `allow LIST Order when hasEdge("orderShipsToRegion", request.region)`

Under the hood, policy evaluation uses `ListQueryRewriter.hasEdge(...)`, which converts `hasEdge` into a Morphia Filter limiting `_id` to the allowed source ids; compose it with your base filter via `Filters.and(...)`.

4.3. Passing tenantId correctly

- Always resolve `tenantId` from `RuleContext/SecurityContext` (the same source your repos use for realm/database selection).
- `EdgeDao` and `ListQueryRewriter` already accept `tenantId`; never cross tenant boundaries when reading edges.
- Index recommendation (per tenant):
 - `(tenantId, p, dst)`
 - `(tenantId, src, p)`

4.4. Morphia repository integration patterns

The goal is zero-friction usage in existing repos without invasive changes.

Option A: Apply ontology constraints in code paths that already construct BSON filters.

- If your repo method builds a Bson filter before calling `find()`, wrap it through `rewriter`:

```
Bson base = Filters.and(existingFilter1, existingFilter2);
Bson rewritten = hasEdgeRequested
    ? rewriter.rewriteForHasEdge(base, tenantId, predicate, dst)
    : base;
var cursor = datastore.getDatabase().getCollection(coll).find(rewritten);
```

Option B: Apply ontology constraints to Morphia Filter/Query via ids.

- When the repo uses Morphia's typed query API instead of BSON, pre-compute the id set and constrain by `_id`:

```
Set<String> ids = edgeDao.srcIdsByDst(tenantId, predicate, dst);
if (ids.isEmpty()) {
    return List.of(); // short-circuit
}
query.filter(Filters.in("_id", ids));
```

Option C: Centralize in a tiny helper for developer ergonomics.

- Provide one helper in your application layer, invoked wherever policies inject additional constraints:

```
public final class OntologyFilterHelper {
```

```
private final ListQueryRewriter rewriter;
public OntologyFilterHelper(ListQueryRewriter r) { this.rewriter = r; }

public Bson ensureHasEdge(Bson base, String tenantId, String predicate, String dst)
{
    return rewriter.rewriteForHasEdge(base, tenantId, predicate, dst);
}
}
```

4.5. Where ontology materialization happens

- Automatic on write via repository hook:
- When you save or update an entity, the framework runs an internal `PostPersistHook` (`OntologyWriteHook`) that:
 - extracts explicit edges from fields/getters annotated with `@OntologyProperty` using a startup-built metadata cache (no per-call reflection), and
 - calls `OntologyMaterializer` to infer inverse, transitive, symmetric, and super-property edges and upsert them to the edges collection.
- This is enabled by default. To disable temporarily (e.g., for bulk loads), set: `ontology.auto-materialize=false`

4.6. Custom Edge Providers: `OntologyEdgeProvider` and `HierarchyListEdgeProvider`

While annotations (`@OntologyProperty`) and property chains cover many use cases, some relationships require custom logic to determine edges. Quantum provides an SPI for implementing custom edge providers that are automatically discovered and integrated into the materialization process.

4.6.1. `OntologyEdgeProvider`: The Core SPI

The `OntologyEdgeProvider` interface allows you to contribute explicit edges for any entity instance. These edges are merged with annotation-derived edges before being passed to the reasoner.

```
public interface OntologyEdgeProvider {
    /** @return true if this provider supports the given entity type */
    boolean supports(Class<?> entityType);

    /** Produce explicit edges for the given entity instance */
    List<Reasoner.Edge> edges(String realmId, DataDomainInfo dataDomainInfo, Object
entity);
}
```

Implementations should be annotated with `@ApplicationScoped` for CDI discovery.

4.6.2. HierarchyListEdgeProvider: Handling Complex Hierarchies

A common pattern in SaaS applications is assigning users/entities to nodes in a hierarchy (e.g., Territories, Organizations) where they should also "inherit" relationships from descendant nodes. `HierarchyListEdgeProvider` is a specialized base class designed for this purpose.

Key features: - **Hierarchy Traversal**: Automatically expands relationships through parent-child links. - **List Resolution**: Resolves `StaticDynamicList` fields to concrete entity IDs. - **Change Tracking**: Informs the framework which entities need re-materialization when a hierarchy node or list changes.

Example: Territory → Location Materialization

Consider a scenario where: 1. **Associates** are assigned to **Territories**. 2. **Territories** form a hierarchy. 3. Each **Territory** has a list of **Locations**. 4. We want to materialize `Associate --canAccessLocation-> Location` edges.

```
@ApplicationScoped
public class TerritoryLocationEdgeProvider
    extends HierarchyListEdgeProvider<Territory, Territory, Location> {

    @Inject TerritoryRepo territoryRepo;
    @Inject LocationListResolver listResolver;

    @Override public Class<Territory> getSourceType() { return Territory.class; }
    @Override public String getPredicate() { return "hasLocation"; }
    @Override public String getTargetTypeName() { return "Location"; }

    @Override
    protected List<String> getAssignedNodeIds(Territory source) {
        return List.of(source.getId().toString());
    }

    @Override
    protected List<Territory> getChildNodes(ComputationContext context, String nodeId)
    {
        return territoryRepo.getAllChildren(context.dataDomainInfo(), nodeId);
    }

    @Override
    protected List<Location> resolveListItems(ComputationContext context, Territory
node) {
        return listResolver.resolve(context.dataDomainInfo(), node
.getStaticDynamicList());
    }

    @Override
    protected String extractTargetId(Location target) {
        return target.getRefName();
    }
}
```

```
}
```

Combined with a Property Chain:

```
chains:
- chain: [hasTerritory, hasLocation]
  implies: canAccessLocation
```

This setup ensures that when an Associate is linked to a Territory, they automatically get `canAccessLocation` edges to all locations in that territory and its descendants.

4.6.3. When to use Property Chains vs ComputedEdgeProvider

Understanding when to use property chains versus `ComputedEdgeProvider` is critical for modeling your ontology effectively. This section provides clear guidance on which approach to use.

Property Chains: Use for Sequential Multi-Hop Traversals

Property chains are ideal when your derived relationship follows a **fixed sequence of predicates** where each step is a simple, direct relationship.

Property chains handle:

- 1. Multi-hop paths through direct predicates:** Deriving relationships through intermediate entities
 - Example: `Order --placedBy→ Customer --memberOf→ Organization ⇒ Order --placedInOrg→ Organization`
 - Example: `Order --orderHasShipment→ Shipment --shipsTo→ Address ⇒ Order --orderShipsTo→ Address`
- 2. Transitive closure within chains:** When one step is a transitive property
 - Example: `Order --placedInOrg→ Org1 --ancestorOf→ Org2 ⇒ Order --placedInOrg→ Org2`
 - Propagates derived relationships up hierarchies automatically
- 3. Chain composition:** Output of one chain feeds another
 - Chain 1: `[orderHasShipment, shipsTo] ⇒ orderShipsTo`
 - Chain 2: `[orderShipsTo, locatedIn] ⇒ orderShipsToRegion`

Property chain example (YAML):

```
chains:
- chain: [placedBy, memberOf]
  implies: placedInOrg
- chain: [orderHasShipment, shipsTo]
  implies: orderShipsTo
- chain: [orderShipsTo, locatedIn]
```

`implies: orderShipsToRegion`

Property chain limitations:

- Chains must be **regular** (non-transitive predicates in the chain itself)
- Chains are **strict sequential paths** - no branching, alternation (**OR**), or conditional logic
- No backtracking or negation
- Cannot involve **external data lookups** or **dynamic queries** during computation
- Cannot handle **collection expansion** where one entity has a list that must be resolved

ComputedEdgeProvider: Use for Complex Logic Beyond Sequential Paths

Use `ComputedEdgeProvider` when your derived edges require:

1. **Database queries during computation:** Loading related entities to determine targets
2. **Collection/list resolution:** The target set comes from resolving a list (static or dynamic)
3. **Conditional logic:** Different targets based on entity state or business rules
4. **Non-sequential traversals:** Logic that doesn't follow a simple $A \rightarrow B \rightarrow C$ pattern

When to use ComputedEdgeProvider:

Scenario	Why Property Chains Won't Work
Associate → canSeeLocation → Location (via Territory hierarchy + LocationList)	Requires loading the Territory, expanding to children, then resolving each Territory's LocationList - involves list resolution
User → canAccessCustomer → Customer (via assigned Regions and dynamic filters)	Requires executing dynamic queries based on filter strings stored in Region entities
Manager → canViewReport → Report (where Reports are determined by Department + Role combinations)	Requires conditional logic based on both hierarchy position AND role assignments
Entity → derivedTarget → Target (where targets depend on runtime configuration)	Requires external lookups that chains cannot perform

HierarchyListEdgeProvider: Specialized for Hierarchy + List Pattern

`HierarchyListEdgeProvider<S, H, T>` is a specialized base class for the common pattern:

1. Get assigned hierarchy nodes from source entity (e.g., Associate's Territories)
2. Expand to include all child nodes (hierarchy traversal)
3. Resolve each node's list (static or dynamic)
4. Create edges to the resolved items

Use HierarchyListEdgeProvider when:

- Your source entity is assigned to hierarchy nodes (Territories, Departments, Regions)
- Each hierarchy node has an associated list of target entities
- The relationship should include targets from the assigned node AND all descendants
- You need provenance tracking of which nodes and lists contributed to each edge

Example use cases:

- `Associate` → `canSeeLocation` → `Location` via Territory hierarchy
- `SalesRep` → `canAccessAccount` → `Account` via Region hierarchy
- `Employee` → `canViewDocument` → `Document` via Department hierarchy

Example implementation:

```
@ApplicationScoped
public class AssociateCanSeeLocationProvider
    extends HierarchyListEdgeProvider<Associate, Territory, Location> {

    @Inject TerritoryRepo territoryRepo;
    @Inject LocationListResolver listResolver;

    @Override protected Class<Associate> getSourceType() { return Associate.class; }
    @Override protected String getPredicate() { return "canSeeLocation"; }
    @Override protected String getTargetTypeName() { return "Location"; }
    @Override protected String getHierarchyTypeName() { return "Territory"; }

    @Override
    protected List<String> getAssignedNodeIds(Associate source) {
        return source.getAssignedTerritories().stream()
            .map(ref -> ref.getId().toString())
            .toList();
    }

    @Override
    protected Optional<Territory> loadHierarchyNode(ComputationContext ctx, String
nodeId) {
        return territoryRepo.findById(new ObjectId(nodeId));
    }

    @Override
    protected List<Territory> getChildNodes(ComputationContext ctx, String nodeId) {
        return territoryRepo.getAllChildren(new ObjectId(nodeId));
    }

    @Override
    protected List<Location> resolveListItems(ComputationContext ctx, Territory node)
{
        return listResolver.resolve(ctx.getDataDomainInfo(), node.
getStaticDynamicList());
    }
}
```

```

    }

    @Override
    protected String extractTargetId(Location target) {
        return target.getId().toString();
    }
}

```

Decision Guide

Approach	When to Use	Example
Property Chain (YAML)	Sequential multi-hop through direct predicates; no external data lookups needed	<code>Order --placedBy→ Customer</code> <code>--memberOf→ Org</code>
Property Chain with Transitive	Hierarchy rollup where relationships are direct predicates	<code>Order --placedInOrg→ Org</code> <code>--ancestorOf→ ParentOrg</code>
ComputedEdgeProvider	Complex logic, conditional branching, or external data lookups	Permission edges based on roles + entity state
HierarchyList EdgeProvider	Source assigned to hierarchy nodes, each with associated target lists	<code>Associate → canSeeLocation → Location</code> via Territory + LocationList

Important Notes

- Source types must be `@OntologyClass` annotated:** `ComputedEdgeProvider` implementations will show a warning at startup if their source type lacks `@OntologyClass`. The edges will not be processed during entity persistence.
- Providers are automatically discovered:** Any `@ApplicationScoped` class extending `ComputedEdgeProvider` is automatically discovered and registered at startup.
- Provenance is tracked:** `ComputedEdgeProvider` and `HierarchyListEdgeProvider` automatically track which inputs contributed to each computed edge, enabling smart incremental updates.
- Combine approaches when needed:** You can use property chains for some relationships and computed providers for others in the same application.

4.6.4. Extending materialization with `ComputedEdgeProvider` (Advanced)

While `OntologyEdgeProvider` is sufficient for simple, direct edges, many real-world use cases require edges computed from complex logic, such as hierarchy traversal or dynamic list resolution. `ComputedEdgeProvider` is a more robust base class that provides:

- Provenance tracking:** Records why an edge exists (which hierarchy nodes and lists were involved).
- Incremental updates:** Tracks dependencies to know exactly which edges to recompute when a source changes.
- Specialized hierarchy support:** `HierarchyListEdgeProvider` simplifies implementing common

patterns like "User can see Locations in their assigned Territories (and descendants)".

Example: Hierarchy-based Edge Provider

```
@ApplicationScoped
public class AssociateCanSeeLocationProvider
    extends HierarchyListEdgeProvider<Associate, Territory, Location> {

    @Inject TerritoryRepo territoryRepo;
    @Inject LocationListResolver listResolver;

    @Override protected Class<Associate> getSourceType() { return Associate.class; }
    @Override protected String getPredicate() { return "canSeeLocation"; }
    @Override protected String getTargetTypeName() { return "Location"; }
    @Override protected String getHierarchyTypeName() { return "Territory"; }

    @Override
    protected List<String> getAssignedNodeIds(Associate source) {
        return source.getAssignedTerritories().stream()
            .map(ref -> ref.getId().toString())
            .toList();
    }

    @Override
    protected Optional<Territory> loadHierarchyNode(ComputationContext ctx, String
nodeId) {
        return territoryRepo.findById(new ObjectId(nodeId));
    }

    @Override
    protected List<Territory> getChildNodes(ComputationContext ctx, String nodeId) {
        return territoryRepo.getAllChildren(new ObjectId(nodeId));
    }

    @Override
    protected List<Location> resolveListItems(ComputationContext ctx, Territory node)
{
        return listResolver.resolve(ctx.getDataDomainInfo(), node.
getStaticDynamicList());
    }

    @Override
    protected String extractTargetId(Location target) {
        return target.getId().toString();
    }
}
```

Multi-hop and Provenance

`ComputedEdgeProvider` supports multi-hop capabilities by allowing providers to contribute to the overall ontology graph that is materialized. The provenance information captured by these providers includes:

- **Hierarchy Contributions:** Every node traversed in a hierarchy path.
- **List Contributions:** Every list (static or dynamic) resolved to find target entities.

This provenance is stored with the edge and allows the framework to perform smart, incremental recomputations when underlying data (like a Territory's child nodes or a LocationList's filter) changes.

4.6.5. Extending materialization with `OntologyEdgeProvider` (SPI)

The write hook is annotation-first and SPI-second:

- First, it extracts explicit edges from `@OntologyProperty` fields/getters on the entity being saved.
- Then, it discovers any CDI beans implementing `com.e2eq.ontology.spi.OntologyEdgeProvider` that support the entity type and merges their edges.
- Finally, it persists explicit edges, prunes stale ones, and rematerializes inferred edges (inverse, `subPropertyOf`, transitive, symmetric).

This keeps `OntologyWriteHook` thin and generic while allowing domain modules to contribute additional explicit edges without changing the core.

Basic contract

- Implementations must declare:
- `boolean supports(Class<?> entityType)` — return true for the entity types you handle.
- `List<Reasoner.Edge> edges(String realmId, Object entity)` — return explicit edges to add for that instance.
- Register the implementation as a CDI bean (e.g., `@ApplicationScoped`) in your module; Quarkus will discover it.
- Edges you return should use your domain `refName` (or `id`) for `src/dst` ids and the ontology type ids for `srcType/dstType`.

Example provider

```
@jakarta.enterprise.context.ApplicationScoped
public class UserCredentialEdgeProvider implements com.e2eq.ontology.spi
.OntologyEdgeProvider {
    @Override
    public boolean supports(Class<?> entityType) {
        return com.e2eq.framework.model.security.User.class.isAssignableFrom(entityType);
    }
}
```

```

@Override
public java.util.List<com.e2eq.ontology.core.Reasoner.Edge> edges(String realmId,
Object entity) {
    var user = (com.e2eq.framework.model.security.User) entity;
    String srcId = user.getRefName();
    if (srcId == null || srcId.isBlank()) return java.util.List.of();
    String credRef = user.getCredentialRefName();
    if (credRef == null || credRef.isBlank()) return java.util.List.of();
    return java.util.List.of(
        new com.e2eq.ontology.core.Reasoner.Edge(
            srcId, "User", "userHasCredential", credRef, "CredentialUserIdPassword",
            false, java.util.Optional.empty()
        )
    );
}
}

```

Merging behavior and lifecycle

- The hook calls all matching providers and concatenates their edges with the annotation-derived ones.
- It computes the diff against prior explicit edges for the same source and prunes any stale explicit edges per predicate.
- Then it invokes the reasoner/materializer to compute and upsert inferred edges.
- Finally, it executes cascade policies (e.g., ORPHAN_REMOVE/DELETE) declared on `@OntologyProperty`, using the prior vs. new explicit state.

ID and type resolution tips

- `srcId`: By default, the hook resolves an entity id using `getRefName()`, `getId()`, or `toString()` fallback. Prefer setting `refName` on your entities.
- `dstId`: You may point to another entity by `refName` string (no fetch required) if your domain guarantees uniqueness per type and tenant.
- `srcType/dstType`: Use the `@OntologyClass(id=...)` values for the types (e.g., "User", "CredentialUserIdPassword").
- `realmId`: The framework passes the current tenant/realm id; include it when persisting or resolving external references if needed.

Testing your provider

- You can write a Quarkus test that persists a source and any targets, calls `OntologyWriteHook.afterPersist(tenant, entity)`, and asserts that:
 - the annotation-derived edges exist, and
 - your provider-derived edges are also present and marked as explicit (`inferred=false`).

Migration guidance

- Start annotation-first. Only add a provider when edges depend on computed fields, cross-aggregate ids, or non-entity sources.
- Keep providers small and focused per entity type. Avoid duplicating edges that annotations already cover.
- If multiple providers contribute the same edge, the repo upsert is idempotent.
- Intermediates and re-materialization:
 - If you change entities that are not the source but affect chains (e.g., Address.region, Customer.memberOf), those sources will get updated the next time they are saved. For immediate consistency across a tenant, run the re-materialization job (see below).
- Provide nightly/backfill jobs for recomputing edges across a tenant when ontology rules evolve.

4.6.6. Configuration

```
# default is true
ontology.auto-materialize=true
```

4.6.7. Example: annotate and save — edges are materialized automatically

```
@OntologyClass(id = "Order")
public class Order {
    private String refName;

    @OntologyProperty(id = "placedBy", functional = true)
    public Customer getCustomer() { return customer; }
}

@OntologyClass(id = "Customer")
public class Customer {
    private String refName;

    @OntologyProperty(id = "memberOf")
    public Organization getOrg() { return org; }
}
```

- Saving an Order will automatically create explicit placedBy and, via chains/inference, placedInOrg and inverse edges as defined in your ontology registry. No manual calls to OntologyMaterializer.apply(...) are required.

4.7. Security and multi-tenant considerations

- Edge rows include tenantId and should be validated/filtered by tenant on every operation.
- Never trust a client-supplied predicate or destination id blindly; combine with rule evaluation and whitelist allowed predicates per domain if needed.

- For shared resources across tenants (rare), model cross-tenant permissions at the policy layer; don't reuse edges across tenants unless explicitly designed.

4.8. Provisional vs. admitted vocabulary tier

Declaring a predicate in the ontology (the TBox) does not by itself authorize it for use in security rules. There are two tiers:

- **Provisional** — a predicate declared in the active TBox's `properties`. It is usable for facts and materialized edges, but a security rule that references it (for example `hasEdge("placedInOrg", ...)`) is rejected.
- **Admitted** — a predicate that has been explicitly admitted for policy use. Only admitted predicates may be referenced by security rules.

The realm-governance set lives on `TenantOntologyTBox.admittedPredicates` (quantum-ontology-mongo) and is three-valued:

<code>admittedPredicates</code>	Meaning
<code>null</code>	Legacy realm: all declared predicates are admitted . This is the back-compatible default — realms that predate the tier are unaffected.
empty set	Governed realm that admits no predicates for rule use; every declared predicate is provisional.
populated set	Governed realm: exactly these predicates may be referenced by rules; everything else declared is provisional.

Enforcement happens at rule-registration time. When `RuleContext` adds a rule it fires a `RuleVocabularyCheck`; `PolicyVocabularyGuard` (quantum-ontology-policy-bridge) observes it and runs `RuleVocabularyValidator` over the predicate literals the rule references. A reference to an undeclared predicate (`ABSENT`) or a declared-but-not-admitted predicate (`PROVISIONAL`) fails rule registration, naming the rule and the offending predicate.

This is fail-closed by design. The disposition is read through the three-state `AdmittedVocabularyResult` (`LEGACY` / `GOVERNED` / `UNAVAILABLE`); if the admitted-set read fails (for example a transient Mongo outage) the result is `UNAVAILABLE` and the guard refuses to downgrade a governed realm to "all admitted" — rule registration that references any predicate is rejected rather than silently re-opening the vocabulary.



Admitting a predicate is an admin, audited, human-in-the-loop action. Promotion and demotion are performed through the enterprise ontology-governance control plane (`POST /v1/ontology/{realm}/vocabulary/{predicate}:promote` and `...:demote`), which also owns TBox admission. The first promote/demote on a legacy realm grandfathers the currently-declared predicates so no existing rule is silently de-authorized. See the Quantum Enterprise book's **Ontology governance control plane** chapter for the admission and vocabulary endpoints, the canonical TBox hash version token, and the control-plane / data-plane split.

4.9. Developer workflow and DX checklist

- When writing a rule: use `hasEdge("<predicate>", <rhs>)` and rely on `RuleContext` variables for the destination when possible.
- When writing a list endpoint: read optional ontology filter hints from the policy layer; if present, apply `ensureHasEdge(...)` before `find()`.
- When changing domain relationships: update predicates/chains and re-materialize; list/policy code stays unchanged.
- When indexing a new tenant: include the edges indexes early and validate via a smoke test query using `ListQueryRewriter`.

4.10. Cookbook: end-to-end example with Orders + Org

- Policy: allow LIST Order when `hasEdge("placedInOrg", principal.orgRefName)`
- Request lifecycle: 1) Security filter builds `SecurityContext` and `RuleContext` with `tenantId` and `principal`. 2) Policy evaluation returns a directive to constrain by `hasEdge("placedInOrg", orgRefName)`. 3) Repo builds base filter (`state != ARCHIVED`, etc.). 4) Repo calls `OntologyFilterHelper.ensureHasEdge(base, tenantId, "placedInOrg", orgRefName)`. 5) Mongo executes a single-hop query using materialized edges; results respect both policy and multi-tenancy.

4.11. Migration notes for teams using @Reference

- Keep existing references for write-side integrity and local joins where simple.
- Introduce ontology edges on hot read paths first; update policy rules to `hasEdge` and verify results.
- Gradually replace deep `$lookup` traversals with `hasEdge`-based rewrites.
- Ensure materialization hooks are deployed before removing data fields used as inputs to the ontology.

Chapter 5. Primer: First-time guide to core relationship semantics

If you are new to ontology terms, the following quick explanations will help you build intuition. Each concept explains what it means, a tiny example, and how it affects queries using `hasEdge`.

- **Functional property**
 - What it means: For a given source entity, there is at most one target for this property. Think “single-valued.”
 - Example: `placedBy(Order, Customer)` is functional because an `Order` is placed by exactly one `Customer`.
 - Why it matters: On writes, the latest value replaces any previous one. In storage, you can enforce uniqueness for `(tenantId, src, p)`. Queries are simpler because there’s at most one matching edge per source.
 - In practice: When you save an `Order` with a different `placedBy`, the old edge is replaced so `hasEdge("placedBy", customerX)` reflects the latest truth.
- **Transitive property**
 - What it means: The relationship “chains through” intermediates. If `A` relates-to `B` and `B` relates-to `C` with the same property `p`, then `A` relates-to `C` by `p` as well.
 - Example: `ancestorOf(Org, Org)` is transitive: `OrgA ancestorOf OrgB` and `OrgB ancestorOf OrgC` implies `OrgA ancestorOf OrgC`.
 - Why it matters: You can answer reachability questions without specifying every hop. The system can materialize or compute the closure so queries like `hasEdge("ancestorOf", OrgC)` return `OrgA` even if the path is multiple hops.
 - In practice: Use transitive for hierarchies like `parent/ancestor`, `partOf`, `locatedIn`, `reportsTo`.
- **subPropertyOf (property hierarchy)**
 - What it means: One property is a more specific form of another. If $p \sqsubseteq q$ (“`p` is a sub-property of `q`”), then every `(s, p, o)` also counts as `(s, q, o)`.
 - Example: `placedInOrg` $\sqsubseteq$ `inOrg`. If an `Order` is `placedInOrg ACME`, then it is also `inOrg ACME`.
 - Why it matters: Queries written against the broader property (`q`) automatically include results from all its specializations (`p`). The system may materialize the super-property edges for performance.
 - In practice: You can write policies/queries once for `inOrg` and still match edges stored as `placedInOrg`.
- **inverseOf (inverse properties)**
 - What it means: Two properties point in opposite directions. If `p` is the inverse of `q`, then `(s, p, o)` implies `(o, q, s)`.
 - Example: `parentOf` $\sqsubseteq$ `childOf`. If `OrgA parentOf OrgB`, then `OrgB childOf OrgA`.
 - Why it matters: You only need to assert one direction; the system can infer the other. Queries

can use whichever direction is natural.

- In practice: If you stored `parentOf`, you can still query with `hasEdge("childOf", OrgA)` and find `OrgB` once inverses are materialized.

Putting it together - These traits combine. For example, `placedInOrg` can be declared a `subPropertyOf inOrg`; `ancestorOf` can be transitive; `parentOf` and `childOf` can be inverses. The reasoner uses these facts to materialize convenient edges so your queries remain simple and fast. - Your application code typically continues to use the same three query helpers: `hasEdge`, `hasEdgeAny`, `notHasEdge`. The richer semantics affect which edges exist, not how you call them.

Chapter 6. New ontology features in this release

This release expands the ontology engine and query integration to support a richer, OWL-RL-inspired subset while keeping the developer experience simple and backward compatible. The following capabilities are now available and used transparently by the materializer and query rewriter:

6.1. Feature overview

- `subClassOf` (class hierarchy)
- Model taxonomies (e.g., `Order` $\sqsubset$ `Entity`, `Employee` $\sqsubset$ `Person`). Used for validation and optional type filters.
- `subPropertyOf` (property hierarchy)
- If $p \sqsubset q$ and you assert (s, p, o) , the system materializes (s, q, o) . Queries on q will match p 's edges as well.
- inverse properties
- If p is the inverse of q , asserting (s, p, o) will materialize (o, q, s) . Example: `parentOf` $\sqsubset$ `childOf`.
- symmetric properties
- For symmetric p , asserting (s, p, o) materializes (o, p, s) . Example: `peerOf` between organizations.
- transitive properties
- For transitive p , if (s, p, m) and (m, p, o) then (s, p, o) is inferred. Typical for `ancestorOf`, `partOf`, `locatedIn`, `memberOf` within hierarchical contexts.
- functional properties
- For functional p , each source s has at most one destination o . Upserts replace the prior value. Useful for single-valued relations like `placedBy` on `Order`.

All inferred edges carry `inferred=true` and provenance containing the rule used (e.g., transitive, inverse, `subPropertyOf`). These edges live in the same collection and are fully queryable.

6.2. Business problems solved

- E-commerce organizational access
- Problem: “Show me all orders placed in ACME (including subsidiaries).”
- Solution: Define `placedBy`, `memberOf`, `placedInOrg` with chain `placedBy` $\sqsubset$ `memberOf` $\Rightarrow$ `placedInOrg`, and make `ancestorOf` transitive. Either materialize closure with a chain `placedInOrg` $\sqsubset$ `ancestorOf` $\Rightarrow$ `placedInOrg` or use p being transitive. Queries use `hasEdge("placedInOrg", orgId)`.
- HR management

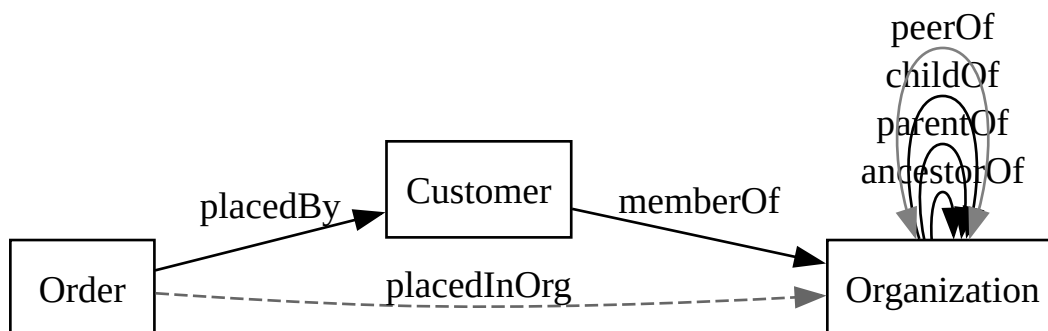
- Problem: “List employees who report to VP_X directly or indirectly.”
- Solution: Define reportsTo as transitive. Query hasEdge("reportsTo", "VP_X").
- Third-party and data lineage
- Problem: “Which datasets are derived from Source S?”
- Solution: Define derivedFrom as transitive and inverse derivedInto. Record symmetric peerOf for bidirectional partnerships where relevant.
- Supplier networks (supply chain)
- Problem: “Find parts supplied by a vendor’s parent company’s peers.”
- Solution: Use subPropertyOf to roll up specialized edges into a common supplies predicate, define parentOf/childOf inverses, and peerOf symmetric at the org level.

6.3. How to code it with current APIs

Below are concise examples showing how to define the ontology, materialize inferences, and query with existing components. These mirror the types already present in this repository and require no grammar changes.

6.3.1. 1) Define ontology (programmatic TBox)

A small TBox-level view of the classes and predicates we define:



Java (at startup, e.g., in a CDI producer):

```

@Produces @Singleton
public OntologyRegistry ontologyRegistry() {
    Map<String, OntologyRegistry.ClassDef> classes = Map.of(
        "Order",      new OntologyRegistry.ClassDef("Order", Set.of(), Set.of(), Set.
of()),
        "Customer",   new OntologyRegistry.ClassDef("Customer", Set.of(), Set.of(),
Set.of()),
        "Organization", new OntologyRegistry.ClassDef("Organization", Set.of(), Set.of(),
Set.of())
    );
}
  
```

```

Map<String, OntologyRegistry.PropertyDef> props = new HashMap<>();
// Single-valued ⇒ functional
props.put("placedBy", new OntologyRegistry.PropertyDef(
    "placedBy", Optional.of("Order"), Optional.of("Customer"), false, Optional.
empty(),
    false, /*transitive*/ false, /*symmetric*/ true /*functional*/, Set.of()
));
props.put("memberOf", new OntologyRegistry.PropertyDef(
    "memberOf", Optional.of("Customer"), Optional.of("Organization"), false,
Optional.empty(),
    false, false, false, Set.of()
));
props.put("placedInOrg", new OntologyRegistry.PropertyDef(
    "placedInOrg", Optional.of("Order"), Optional.of("Organization"), false,
Optional.empty(),
    false, false, false, Set.of("inOrg") // subPropertyOf example
));
props.put("inOrg", new OntologyRegistry.PropertyDef(
    "inOrg", Optional.of("Order"), Optional.of("Organization"), false, Optional
.empty(),
    false, false, false, Set.of()
));
// Transitive ancestor relation
props.put("ancestorOf", new OntologyRegistry.PropertyDef(
    "ancestorOf", Optional.of("Organization"), Optional.of("Organization"), false,
Optional.empty(),
    true, /*transitive*/ false, false, Set.of()
));
// Inverses and symmetric
props.put("parentOf", new OntologyRegistry.PropertyDef(
    "parentOf", Optional.of("Organization"), Optional.of("Organization"), false,
Optional.of("childOf"),
    false, false, false, Set.of()
));
props.put("childOf", new OntologyRegistry.PropertyDef(
    "childOf", Optional.of("Organization"), Optional.of("Organization"), true,
Optional.empty(),
    false, false, false, Set.of()
));
props.put("peerOf", new OntologyRegistry.PropertyDef(
    "peerOf", Optional.of("Organization"), Optional.of("Organization"), false,
Optional.empty(),
    false, /*transitive*/ true, /*symmetric*/ false, /*functional*/ Set.of()
));

List<OntologyRegistry.PropertyChainDef> chains = List.of(
    // Order --placedBy--> Customer --memberOf--> Org ⇒ Order --placedInOrg--> Org
    new OntologyRegistry.PropertyChainDef(List.of("placedBy", "memberOf"),
    "placedInOrg"),
    // Include ancestor closure for placedInOrg results

```

```

        new OntologyRegistry.PropertyChainDef(List.of("placedInOrg", "ancestorOf"),
        "placedInOrg")
    );

    return OntologyRegistry.inMemory(new OntologyRegistry.TBox(classes, props, chains));
}

```

Notes: - subPropertyOf is modeled via the PropertyDef.subPropertyOf set. - Inverse and symmetric are encoded in PropertyDef.inverseOf and PropertyDef.symmetric. - Transitivity is toggled via PropertyDef.transitive.

6.3.2. 2) Materialize inferences when data changes

Use OntologyMaterializer or call the Reasoner directly and upsert edges via OntologyEdgeRepo.

```

@Inject ForwardChainingReasoner reasoner;
@Inject OntologyRegistry ontologyRegistry;
@Inject OntologyEdgeRepo edgeRepo;

public void onOrderSaved(String tenant, String orderId, String customerId, String
orgId) {
    List<Reasoner.Edge> explicit = List.of(
        new Reasoner.Edge(orderId, "Order", "placedBy", customerId, "Customer", false,
Optional.empty()),
        new Reasoner.Edge(customerId, "Customer", "memberOf", orgId, "Organization",
false, Optional.empty())
    );
    Reasoner.EntitySnapshot snap = new Reasoner.EntitySnapshot(tenant, orderId, "Order",
explicit);
    Reasoner.InferenceResult out = reasoner.infer(snap, ontologyRegistry);
    for (Reasoner.Edge e : out.addEdges()) {
        Map<String, Object> prov = e.prov().map(p -> Map.<String, Object>.of("rule", p.rule(
), "inputs", p.inputs())).orElse(Map.of());
        edgeRepo.upsert(tenant, e.srcType(), e.srcId(), e.p(), e.dstType(), e.dstId(),
true, prov);
    }
}

```

The materializer computes and persists: - subPropertyOf roll-ups (e.g., placedInOrg ⊆ inOrg ⇒ write inOrg) - inverse/symmetric counterparts - transitive closures (bounded to the snapshot inputs; you can also chain for closures)

6.3.3. 3) Query with ListQueryRewriter (Quarkus/Morphia)

Use hasEdge/hasEdgeAny/notHasEdge to turn ontology constraints into Morphia filters over your entity collection. The rewriter internally fetches the set of source ids from the edge store and builds an in("refName", ...) filter that composes with your attribute predicates.

```

@Inject ListQueryRewriter queryRewriter;
@Inject MorphiaDatastore datastore;

public List<TestOrder> openOrdersIn(String tenant, String orgId) {
    var status = dev.morphia.query.filters.Filters.eq("status", "OPEN");
    var inOrg = queryRewriter.hasEdge(tenant, "placedInOrg", orgId);
    var combined = dev.morphia.query.filters.Filters.and(status, inOrg);
    return datastore.find(TestOrder.class).filter(combined).iterator().toList();
}

```

To include multiple orgs:

```

var f = queryRewriter.hasEdgeAny(tenant, "placedInOrg", List.of("ORG-ACME", "ORG-
GLOBEX"));

```

To exclude restricted orgs:

```

var f = queryRewriter.notHasEdge(tenant, "placedInOrg", "ORG-RESTRICTED");

```

6.3.4. Functional properties at write time

For single-valued relationships declared functional, configure a unique index over (tenantId, src, p) in your edge store (or rely on repo semantics) and treat upserts as replacements.

Practical guidance: - Mark naturally single-valued links functional (placedBy, primaryOwner). - On write, remove or overwrite any prior dst for the same (tenant, src, p).

6.4. Backward compatibility and migration

- Grammar/API: No changes needed. The same hasEdge/hasEdgeAny/notHasEdge functions work; results simply become more expressive as ontology features are enabled.
- Feature flags: You may enable features per property (e.g., transitive) gradually. Queries remain stable.
- Provenance: Keep prov so you can re-compute or retract inferred edges when inputs or rules change.

6.5. Troubleshooting and tips

- Empty result from hasEdge: Means no matching edges exist; check whether inferences are being materialized for the predicate you expect.
- Tenant scoping: All repo calls include tenantId; ensure you pass the correct tenant in both write and read paths.
- Indexes: For heavy reads, ensure compound indexes on (tenantId, p, dst) and (tenantId, src, p).

For functional, consider a unique (tenantId, src, p).

- Testing: Use Quarkus `@QuarkusTest` to wire CDI and a test `OntologyRegistry` producer. Populate explicit edges, run the reasoner, then assert with the repo and query rewriter as shown in the integration tests included with the repository.

Chapter 7. Model-first ontology with annotations and MorphiaOntologyLoader

This section shows how to use your Morphia model classes as the source of truth for the ontology. You annotate model classes/fields to declare ontology concepts and property traits; at startup, MorphiaOntologyLoader scans MorphiaDatastore.getMapper() to build the OntologyRegistry automatically.

7.1. Why model-first?

- Single source of truth: semantics live next to the data model; less drift between code and config.
- Safer refactors: renames/types evolve in code and the ontology follows.
- No separate YAML/JSON needed for most use cases; optional overrides remain possible later if needed.

7.2. Annotations overview

Two lightweight annotations are used on model classes and fields. They are provided by quantum-ontology-core and can be added to any Morphia-mapped entity (deriving from UnversionedBaseModel).

- @OntologyClass
 - id: optional explicit class id (defaults to the simple class name)
 - subClassOf: optional extra parents (in addition to Java inheritance)
- @OntologyProperty
 - id: optional explicit property id (defaults to the field name)
 - subPropertyOf: property hierarchy roll-up targets
 - inverseOf: name of the inverse property (declare on one side)
 - transitive: whether the property is transitive
 - symmetric: whether the property is symmetric
 - functional: at most one target per source (defaults to true for single-valued fields)
 - domain: override inferred domain (defaults to declaring @OntologyClass)
 - range: override inferred range (defaults to the field/element type)
 - ref: target ontology class id when this field represents a relationship to another ontology type
 - relation: the multiplicity/cardinality for the relationship (NONE, ONE_TO_ONE, ONE_TO_MANY, MANY_TO_ONE, MANY_TO_MANY)
 - edgeType: the edge label used in the ontology graph; if omitted, falls back to id or field name
 - inverseOfEdge: optional inverse edge label if you prefer naming inverses at the edge level
 - materializeEdge: when false, keeps the reference value but skips edge creation (defaults to true)

- cascade: optional cascade policies (NONE by default). Supports ORPHAN_REMOVE, DELETE, and BLOCK_IF_REFERENCED; see below.
- cascadeDepth: limit for recursive cascades (default 1); used by DELETE cascades to bound depth and prevent long chains



functional is inferred true for single-valued fields and false for collections/maps unless you override it.

7.2.1. Relationship semantics on @OntologyProperty

When a field is annotated with @OntologyProperty and declares any of ref, relation, or edgeType, the framework treats it as a relationship and will materialize an ontology edge for it on persist/update.

Key attributes: - ref: Names the target ontology class id. If omitted, the loader attempts to infer it from the Java type of the field or its generic element type. - relation: Declares multiplicity. If not set, it is inferred from the field type: collections imply plural (ONE_TO_MANY), scalars imply singular (MANY_TO_ONE) by convention. - edgeType: Controls the predicate id used to write/read edges. If absent, the property id (or field name) is used. - materializeEdge: Set to false to store the reference value without creating graph edges; traversal and inferences will ignore this property.

Examples:

```
@OntologyClass(id = "Order")
@Entity("orders")
public class Order extends UnversionedBaseModel {
    // Many orders reference one Customer (MANY_TO_ONE); name the edge explicitly
    @OntologyProperty(
        id = "customer",
        ref = "Customer",
        relation = RelationType.MANY_TO_ONE,
        edgeType = "CUSTOMER_OF"
    )
    private String customerId; // or Customer if you store the object
}

@OntologyClass(id = "SCorp")
@Entity("scorps")
public class SCorp extends UnversionedBaseModel {
    // One SCorp authorizes many Resolutions
    @OntologyProperty(
        id = "resolutionIds",
        ref = "Resolution",
        relation = RelationType.ONE_TO_MANY,
        edgeType = "AUTHORIZES_RESOLUTION"
    )
    private List<String> resolutionIds;
}
```

Runtime materialization (conceptual): - (Order) -[CUSTOMER_OF]→ (Customer) - (SCorp) -[AUTHORIZES_RESOLUTION]→ (Resolution)



If ref is provided but edgeType is omitted, the system uses the property id (or field name). Consider using descriptive edge names for durability of queries.

7.2.2. Cascading policies for relationships

Cascading is opt-in and conservative. It controls what happens to related targets and edges when a relationship is modified or a source is deleted.

Supported policies today: - ORPHAN_REMOVE: On update, when an element is removed from a collection (or a singular reference is replaced), the framework deletes the removed target if and only if no other sources still reference it via the same predicate. This is useful for owned child aggregates. - UNLINK: Edges are always pruned to reflect the current snapshot; you can think of this as implicit and always-on for edge hygiene.

Delete-time cascades (also supported): - DELETE: When deleting the source, also delete targets up to cascadeDepth with cycle guards. Targets are deleted only for predicates that declare DELETE on the source side. - BLOCK_IF_REFERENCED: Before deleting a target (either directly or via DELETE cascade), if the target is still referenced by another source via the same predicate, the deletion is skipped (or blocked) to preserve referential integrity.

Example (delete children when parent is deleted, with safety guards):

```
@OntologyClass(id = "Parent")
@Entity("parents")
public class Parent extends UnversionedBaseModel {
    @OntologyProperty(
        id = "children",
        edgeType = "HAS_CHILD",
        ref = "Child",
        relation = RelationType.ONE_TO_MANY,
        cascade = { CascadeType.DELETE, CascadeType.BLOCK_IF_REFERENCED },
        cascadeDepth = 1 // delete only direct children
    )
    private List<Child> children = new ArrayList<>();
}
```

Behavior: - Deleting a Parent removes explicit edges and, with DELETE, deletes its Child documents up to the specified depth. - With BLOCK_IF_REFERENCED, a Child that is still referenced by another Parent is not deleted; only the edge from the deleted Parent is removed. - Cycles are guarded by a visited set; depth prevents long delete chains.

See integration tests in the ontology-mongo module (CascadeDeleteIT) for expected behavior.

Example (owned children with orphan removal):

```

@OntologyClass(id = "Parent")
@Entity("parents")
public class Parent extends UnversionedBaseModel {
    @OntologyProperty(
        id = "children",
        edgeType = "HAS_CHILD",
        ref = "Child",
        relation = RelationType.ONE_TO_MANY,
        cascade = { CascadeType.ORPHAN_REMOVE }
    )
    private List<Child> children = new ArrayList<>();
}

```

Behavior: - Persisting a Parent materializes edges (P --HAS_CHILD--> C) for current children. - Removing a Child from the collection prunes the edge. If no other Parent points to that Child via HAS_CHILD, the Child entity is deleted. - If the Child is still referenced by another Parent, it is retained.

Verification: See integration tests under quantum-ontology-mongo it module (CascadeOrphanRemoveIT) covering both deletion and shared reference cases.

7.2.3. Business benefits of ontology relationships and cascade

- Faster queries and policies: Pre-materialized edges make filtering by relationships a single indexed lookup instead of deep joins or application-side traversals.
- Safer evolution: Edge names and ontology traits remain stable as you refactor field names/types; queries continue to work.
- Cleaner models: Relationship intent lives in one place (@OntologyProperty) rather than being scattered across repos and services.
- Reduced data drift: Automated edge pruning and orphan removal keep the graph consistent with your source-of-truth objects.
- Auditability: Provenance on inferred edges documents why links exist, aiding debugging and compliance.

7.3. Example: Orders placed in an Organization (e-commerce)

Below we implement the same business example covered earlier using model annotations.

```

@OntologyClass(id = "Order")
@Entity(value = "orders")
public class Order extends UnversionedBaseModel {
    private String status;

    // Order --placedBy--> Customer (single-valued => functional)
}

```

```

@OntologyProperty(id = "placedBy", inverseOf = "placed", functional = true)
private Customer placedBy;
}

@OntologyClass(id = "Customer")
@Entity(value = "customers")
public class Customer extends UnversionedBaseModel {
    // Customer --memberOf--> Org (transitive across org tree)
    @OntologyProperty(id = "memberOf", transitive = true)
    private Org memberOf;
}

@OntologyClass(id = "Organization")
@Entity(value = "orgs")
public class Org extends UnversionedBaseModel {
    // Org --parentOf--> Org, with inverse childOf
    @OntologyProperty(id = "parentOf", inverseOf = "childOf")
    private Org parent;

    // Optional symmetric relationship between peers
    @OntologyProperty(id = "peerOf", symmetric = true)
    private Set<Org> peers;
}

```

Optionally, model a super-property to roll up specialized edges:

```

@OntologyClass(id = "Order")
public class Order extends UnversionedBaseModel {
    @OntologyProperty(id = "placedBy", inverseOf = "placed", functional = true)
    private Customer placedBy;

    // Roll-up predicate for org membership of orders; placedInOrg ⇔ inOrg
    @OntologyProperty(id = "placedInOrg", subPropertyOf = {"inOrg"})
    private Org inOrgDirect;
}

```

With these annotations in place, the loader will infer: - Classes: Order, Customer, Organization (+ hierarchy from Java inheritance) - Properties: placedBy (functional), memberOf (transitive), parentOf ⇔ childOf (inverse), peerOf (symmetric) - subPropertyOf: placedInOrg ⇔ inOrg (if declared)

7.4. How the registry is produced (Quarkus CDI)

OntologyCoreProducers wires MorphiaOntologyLoader by default. On startup it scans your models and produces a singleton OntologyRegistry.

```

@ApplicationScoped
public class OntologyCoreProducers {
    @Inject MorphiaDatastore morphiaDatastore;
}

```

```

@Produces @Singleton
public OntologyRegistry ontologyRegistry() {
    try {
        MorphiaOntologyLoader loader = new MorphiaOntologyLoader(morphiaDatastore);
        OntologyRegistry reg = loader.load();
        return isEmpty(reg) ? emptyRegistry() : reg;
    } catch (Throwable t) {
        return emptyRegistry();
    }
}

```

If you need manual control (tests, migrations), you can call the loader directly:

```

MorphiaOntologyLoader loader = new MorphiaOntologyLoader(datastore);
OntologyRegistry registry = loader.load();

```



Keep annotations minimal. Only add traits that aren't obvious from the Java type system (e.g., transitive, symmetric, inverse, subPropertyOf).

7.5. Defining an ontology with YAML (YamlOntologyLoader)

You can also author your ontology in a simple YAML file and load it at runtime using YamlOntologyLoader. This is useful when:

- You want product or ops teams to evolve predicates and chains without changing Java models.
- You need to add an overlay (additional properties or chains) on top of what is discovered from annotations.

YAML structure supported by YamlOntologyLoader:

- classes: list of concept ids
- properties: list of predicate definitions with optional fields
- id: predicate name
- domain: optional class id for the source
- range: optional class id for the target
- inverseOf: optional inverse predicate id (declare on either side; loader wires both)
- transitive: boolean
- symmetric: boolean
- relation: NONE | ONE_TO_ONE | ONE_TO_MANY | MANY_TO_ONE | MANY_TO_MANY (if provided, overrides functional)
- functional: boolean (deprecated in favor of relation; still supported for backward compatibility)
- subPropertyOf: list of super-predicate ids
- chains: list of property chains
- chain: [p1, p2, ...]
- implies: r

Example YAML (full file available in docs at [user-guide/examples/ontology.yaml](#)):

```

# Example ontology.yaml used by YamlOntologyLoader
version: 1
classes:
  - id: Order
  - id: Customer
  - id: Organization

```

```

properties:
  - id: placedBy
    domain: Order
    range: Customer
    inverseOf: placed
    functional: true
  - id: placed
    domain: Customer
    range: Order
    inverseOf: placedBy
  - id: memberOf
    domain: Customer
    range: Organization
  - id: placedInOrg
    domain: Order
    range: Organization
    subPropertyOf: [ "inOrg" ]
  - id: inOrg
    domain: Order
    range: Organization
  - id: parentOf
    domain: Organization
    range: Organization
    inverseOf: childOf
  - id: childOf
    domain: Organization
    range: Organization
    inverseOf: parentOf
  - id: ancestorOf
    domain: Organization
    range: Organization
    transitive: true
  - id: peerOf
    domain: Organization
    range: Organization
    symmetric: true
chains:
  - chain: [ "placedBy", "memberOf" ]
    implies: "placedInOrg"
  - chain: [ "placedInOrg", "ancestorOf" ]
    implies: "placedInOrg"

```

Loading YAML in code:

- Direct usage (plain Java):

```

import com.e2eq.ontology.core.OntologyRegistry;
import com.e2eq.ontology.core.YamlOntologyLoader;

YamlOntologyLoader loader = new YamlOntologyLoader();

```

```
// From classpath resource (place ontology.yaml under src/main/resources)
OntologyRegistry.TBox tbox1 = loader.loadFromClasspath("/ontology.yaml");

// From a file system path
java.nio.file.Path path = java.nio.file.Path.of("config", "ontology.yaml");
OntologyRegistry.TBox tbox2 = loader.loadFromPath(path);

// Build a registry (in-memory) from a TBox
OntologyRegistry registry = new com.e2eq.ontology.core.InMemoryOntologyRegistry(tbox2);
```

- Quarkus auto-wiring (preferred):

OntologyCoreProducers will try to overlay YAML on top of the Morphia-discovered TBox at startup. It looks for one of the following, in order: - A system property `ontology.yaml.path` pointing to the file. - An environment variable `ONTOLOGY_YAML` pointing to the file. - A conventional file `config/ontology.yaml` in the working directory. - A classpath resource `/ontology.yaml`.

If found, it loads the YAML TBox and merges it with the base TBox from annotations using `OntologyMerger`, then validates using `OntologyValidator`.



Keep overlays additive. If you need to override flags (e.g., mark an existing property as transitive), ensure your overlay defines the same predicate id with the desired traits; the merger will apply the union/override semantics where supported.

7.6. Materializing inferences from annotated models

Materialization is automatic on save/update through the repository hook; you do not need to assemble explicit edge lists or call `OntologyMaterializer` manually.

- What happens:
- The framework inspects fields/getters annotated with `@OntologyProperty` and extracts explicit edges.
- The reasoner applies inverse, symmetric, transitive, and `subPropertyOf` traits from the registry and upserts inferred edges.
- When to trigger re-materialization manually:
- After ontology rule changes (e.g., you mark a property transitive or add an inverse), run your re-materialization job to recompute edges for existing entities in a tenant.
- During bulk imports, you can disable auto-materialize and run the job afterward.

Example: disable auto-materialize for a batch, then rebuild

```
ontology.auto-materialize=false
```

```
@Inject OntologyRebuilder rebuilder; // your maintenance utility

public void runBackfill(String tenantId) {
    rebuilder.recomputeTenant(tenantId);
}
```

7.7. Querying using ListQueryRewriter (unchanged)

Your query code does not change. It benefits from materialized edges that now carry richer semantics from annotations.

```
@Inject ListQueryRewriter queryRewriter;
@Inject MorphiaDatastore datastore;

public List<Order> openOrdersIn(String tenant, String orgId) {
    var status = dev.morphia.query.filters.Filters.eq("status", "OPEN");
    var inOrg = queryRewriter.hasEdge(tenant, "placedInOrg", orgId);
    var combined = dev.morphia.query.filters.Filters.and(status, inOrg);
    return datastore.find(Order.class).filter(combined).iterator().toList();
}
```

To include subsidiaries via ancestor/parent relationships, either: - Declare ancestorOf as transitive in the model and materialize a closure, or - Add a property chain placedInOrg $\square$ ancestorOf $\Rightarrow$ placedInOrg in your registry initialization.

7.8. Business case recipes with annotations

- E-commerce org access
- Annotate placedBy (functional), memberOf (transitive), and optionally parentOf (inverse childOf). Materialize placedInOrg via a property chain. Query hasEdge("placedInOrg", org).
- HR reporting lines
- Annotate reportsTo as transitive on Employee; query hasEdge("reportsTo", managerId).
- Supplier networks
- Annotate supplies as a base property; use subPropertyOf to roll up specialized suppliesDirect and suppliesViaSubsidiary to supplies; annotate peerOf on Org as symmetric.

7.9. Troubleshooting

- The registry is empty at runtime
- Ensure your models are discovered by Morphia (annotated with @Entity) and that the Quarkus application initializes MorphiaDatastore before the ontology producer.
- Inverse not applied

- Declare `inverseOf` on one side only; the loader will wire both directions.
- Functional not enforced
- For functional properties, ensure your edge repository enforces unique (`tenantId`, `src`, `p`) or that your write path overwrites prior values.

7.10. Migration strategy

- Start by adding `@OntologyProperty` to the most important relationships; don't attempt to annotate everything at once.
- Keep existing `@Reference` fields if you have them; use edges for query-time semantics and policy integration first.
- Add tests that verify the loader discovered your properties (including inverses/subPropertyOf) and that queries via `hasEdge` behave as expected.

7.11. Modeling guidance: annotate existing id/reference getters (keep API and DB clean)

- Prefer annotating getters that already exist in your schema (String ids or existing `@Reference` fields). Avoid adding embedded objects solely to drive ontology.
- If you do need a synthetic accessor purely for ontology, hide it from API docs and persistence using `@JsonIgnore`, `@Schema(hidden = true)`, and `@dev.morphia.annotations.Transient`.

Recommended pattern: annotate an existing id getter (Swagger unchanged)

```
import com.e2eq.ontology.annotations.OntologyClass;
import com.e2eq.ontology.annotations.OntologyProperty;
import dev.morphia.annotations.Entity;

@Entity
@OntologyClass(id = "Order")
public class Order extends UnversionedBaseModel {
    // Existing field that is already part of your API/DB contract
    private String customerRefName;

    public String getCustomerRefName() { return customerRefName; }
    public void setCustomerRefName(String v) { this.customerRefName = v; }

    // Drives ontology edges without changing Swagger: the getter already exists and
    // returns a String id
    @OntologyProperty(id = "placedBy", functional = true)
    public String customerIdForEdges() { return getCustomerRefName(); }
}
```

Because `AnnotatedEdgeExtractor` treats `CharSequence` targets as ids, annotating a String-returning getter produces edges without introducing new embedded objects. Your Swagger/OpenAPI and

persisted shape remain unchanged.

Optional: use an existing @Reference instead of an id

```
@Entity
@OntologyClass(id = "Order")
public class Order extends UnversionedBaseModel {
    @dev.morphia.annotations.Reference
    private Customer customer; // already in your schema

    @OntologyProperty(id = "placedBy", inverseOf = "placed", functional = true)
    public Customer getCustomer() { return customer; }
}
```

If you must add an ontology-only accessor, hide it

```
import com.fasterxml.jackson.annotation.JsonIgnore;
import io.swagger.v3.oas.annotations.media.Schema;
import dev.morphia.annotations.Transient;

@Entity
@OntologyClass(id = "Order")
public class Order extends UnversionedBaseModel {
    private String orgRefName; // real field

    // Hidden from API docs and persistence, used only to emit an ontology edge
    @JsonIgnore
    @Schema(hidden = true)
    @Transient
    @OntologyProperty(id = "placedInOrg", subPropertyOf = {"inOrg"})
    public String placedInOrgEdge() { return orgRefName; }
}
```

These patterns keep your REST and database contracts clear while allowing ontology materialization to happen automatically on writes.

Chapter 8. Exploring the ontology via REST and JointJS

This section shows how to inspect your ontology (TBox) and your instance relationships (ABox) through a small, read-only REST API and how to render the results in a browser using JointJS. These endpoints are implemented in the server-only `quantum-ontology-rest` module and are intended for diagnostics, admin tooling, and lightweight visualization. The `quantum-ontology-mongo` module contains the persistence and runtime services without exposing endpoints.



For a complete, copy-paste friendly reference of every parameter and more examples, see the module guide at `quantum-ontology-rest/README.md`. The summary below is intentionally concise to keep this document flowing without duplication.

8.1. Base info

- Base path: `/ontology`
- Media type: `application/json`
- Security: Bearer JWT, roles `user` or `admin`
- Multi-tenancy: pass `X-Realm: <tenant-id>` header for ABox queries (and for any repo-backed calls). TBox demos may work without it if your deployment wires an in-memory registry.

Example (replace token and realm):

```
curl -H "Authorization: Bearer $TOKEN" \  
  -H "X-Realm: demo" \  
  http://localhost:8080/ontology/summary
```

8.2. TBox (schema) endpoints

These expose ontology registry metadata and a JointJS-ready schema graph.

- `GET /ontology/registry` — Full TBox snapshot. Depending on your registry producer, classes may be best-effort.
- `GET /ontology/classes` — Lists known classes (often inferred from property domains/ranges).
- `GET /ontology/classes/{name}` — Class details by name.
- `GET /ontology/properties` — Lists properties.
- `GET /ontology/properties/{name}` — Property details by name.
- `GET /ontology/propertyChains` — Lists property chains.
- `GET /ontology/graph/jointjs` — Returns `{ cells: [...] }` for a quick schema visualization in JointJS.
- `GET /ontology/summary` — Small counts summary (classes/properties/chains).

Basic call:

```
curl -H "Authorization: Bearer $TOKEN" \  
http://localhost:8080/ontology/graph/jointjs | jq '.cells | length'
```

8.3. ABox (instances) graph endpoint

Builds a neighborhood graph around a specific instance (src id + type). Useful for investigating relationships and verifying inferences.

- GET /ontology/graph/instances/jointjs
- Headers: X-Realm: <tenant-id> (required)
- Query params:
 - src: source id (required)
 - type: ontology class/type of the source id (required)
 - direction: out | in | both (default: out)
 - p: repeatable predicate filter (optional)
 - limit: max edges to include (default: 200)

Example:

```
curl -H "Authorization: Bearer $TOKEN" \  
-H "X-Realm: demo" \  
"http://localhost:8080/ontology/graph/instances/jointjs?src=01&type=Order&direction=bo  
th&limit=100"
```

Response shape:

```
{ "cells": [] }
```

- Elements (instances) are rectangles with `attrs.root data-type=instance` and `data-class=<Type>`.
- Links are labeled with the predicate. Inferred links are visually distinguished and marked in data:
- Styling: dashed gray stroke (`strokeDasharray: "5,5"`, `stroke: "#9e9e9e"`).
- Metadata: `data.inferred = true`.

8.4. Rendering with JointJS (quick snippet)

```
async function renderJointJs(divId, url) {  
  const res = await fetch(url, { headers: { 'Authorization': 'Bearer ' + window.token,
```

```
'X-Realm': 'demo' } }));
const { cells } = await res.json();
const graph = new joint.dia.Graph();
graph.fromJSON({ cells });
const paper = new joint.dia.Paper({ el: document.getElementById(divId), model:
graph, width: 1400, height: 900, gridSize: 10, drawGrid: true });
// Optional: apply a layout here
}
// Example: schema graph
// renderJointJs('paper', '/ontology/graph/jointjs');
// Example: instance graph around Order 01
// renderJointJs('paper',
'/ontology/graph/instances/jointjs?src=01&type=Order&direction=both');
```



JointJS will honor the attrs/labels we return. You can add a legend in your UI to explain that dashed gray edges are inferred, while solid edges are explicit.

8.5. When to use these endpoints

- During development: inspect whether your ontology registry is loaded as expected and whether property chains/inverses are effective.
- During troubleshooting: confirm that edges exist for the predicates your queries or policies reference (see also [Integrating Ontology](#)).
- For lightweight admin tools: the JointJS payload can be embedded in a simple HTML page to visualize current relationships.

For deep dives, parameter matrices, and additional examples (including large graph considerations), consult [quantum-ontology-rest/README.md](#) in this repository.

Chapter 9. Developer how-to: YAML properties, chains, write-time materialization, reindex, and provenance

This section is a concise, task-focused guide for developers. It shows how to declare properties and chains in YAML, how write-time materialization works and how to toggle it, how to reindex after YAML changes, and how edge provenance is stored and how deletions are handled. Examples use the Order/Shipment/Address/Org model.

9.1. Declare properties and chains in YAML

Place an ontology.yaml in your classpath (src/main/resources) or supply a path via environment variable ONTOLOGY_YAML or system property ontology.yaml.path. The loader supports classes, properties, and property chains.

```
version: 1
classes:
  - id: Order
  - id: Customer
  - id: Organization
  - id: Shipment
  - id: Address
  - id: Region

properties:
  - id: placedBy
    domain: Order
    range: Customer
    functional: true
  - id: memberOf
    domain: Customer
    range: Organization
  - id: orderHasShipment
    domain: Order
    range: Shipment
  - id: shipsTo
    domain: Shipment
    range: Address
  - id: locatedIn
    domain: Address
    range: Region
  - id: ancestorOf
    domain: Organization
    range: Organization
    transitive: true
# implied predicates materialized by chains
- id: placedInOrg
```

```

    domain: Order
    range: Organization
  - id: orderShipsTo
    domain: Order
    range: Address
  - id: orderShipsToRegion
    domain: Order
    range: Region

chains:
  - chain: [placedBy, memberOf]
    implies: placedInOrg
  - chain: [orderHasShipment, shipsTo]
    implies: orderShipsTo
  - chain: [orderShipsTo, locatedIn]
    implies: orderShipsToRegion
  - chain: [placedInOrg, ancestorOf]
    implies: placedInOrg

```

Notes: - Chains are ordered. The last predicate referenced must already exist as a property id. - You may chain an implied property (e.g., placedInOrg) with another property (ancestorOf) to re-imply itself (useful with transitivity on ancestorOf). - Domain and range help with type checking and tooling; they are optional but recommended.

See also: [Defining an ontology with YAML \(YamlOntologyLoader\)](#) for a broader introduction.

9.2. Write-time materialization: what runs and how to toggle

When an entity annotated with `@OntologyClass` is persisted, the framework: - Extracts explicit edges from fields/getters annotated with `@OntologyProperty` on that entity. - Merges any additional explicit edges from SPI providers (beans implementing `com.e2eq.ontology.spi.OntologyEdgeProvider`). - Calls `OntologyMaterializer`, which runs the forward-chaining reasoner and writes the inferred/derived edges. It also prunes stale explicit and inferred edges for that source.

Global toggle

```

# default is true
ontology.auto-materialize=true

```

- Set `ontology.auto-materialize=false` to temporarily disable the write hook (useful for bulk loads or tests).

Per-property toggle

```

@OntologyProperty(id = "orderHasShipment", materializeEdge = false)

```

```
private List<String> shipmentRefNames;
```

- `materializeEdge=false` skips extracting explicit edges for that property only. This does not disable derived edges that might arise from other explicit inputs.

Provider-based edges

- Implement `OntologyEdgeProvider` to contribute explicit edges not backed by fields. The extractor merges these before materialization. See [Where ontology materialization happens](#) for a full example.

9.3. Reindex after YAML changes

If you change `ontology.yaml` (add properties, mark a property transitive, add chains), you should trigger a tenant-wide re-materialization so derived edges reflect the new rules.

REST endpoints (admin-only):

```
# Start reindex for a realm (tenant)
curl -X POST "http://localhost:8080/ontology/admin/reindex?realm=default"

# Check status
curl -s "http://localhost:8080/ontology/admin/reindex/status"
```

Tips: - Reindex discovers all `@OntologyClass` entities via Morphia and re-applies write-time materialization to each instance. - For large tenants, consider running during off-hours. The service streams per class; status shows progress. - If you need to purge previously derived edges before recomputing, use a force mode if exposed by your deployment (some environments wire `runAsync(realm, force=true)`).

Related: Drift-repair job

- For targeted repair, use the drift-repair endpoint to prune unsupported derived edges and re-derive per entity class with pagination:

```
# Run prune+derive for Orders (page size 100)
curl -X POST "http://localhost:8080/ontology/admin/drift-
repair?realm=default&entityClass=com.yourpkg.Order&pageSize=100"

# Continue from a page token
curl -X POST "http://localhost:8080/ontology/admin/drift-
repair?realm=default&entityClass=com.yourpkg.Order&pageToken=6543210abcdef..."

# Only prune (no derive)
curl -X POST "http://localhost:8080/ontology/admin/drift-
repair?realm=default&entityClass=com.yourpkg.Order&derive=false"

# Dry-run (no writes)
```

```
curl -X POST "http://localhost:8080/ontology/admin/drift-repair?realm=default&entityClass=com.yourpkg.Order&dryRun=true"
```

9.4. Edge provenance format and deletions

Storage model (Mongo edges collection): - Each edge document stores: tenantId, src, srcType, p (predicate), dst, dstType, inferred, derived, prov (map), support (list), ts. - Derived edges include structured support:

```
{
  "dataDomain": { "tenantId": "default" },
  "src": "ORD-1", "srcType": "Order",
  "p": "orderShipsToRegion",
  "dst": "US-WEST", "dstType": "Region",
  "inferred": true, "derived": true,
  "prov": { "rule": "chain(orderShipsTo,locatedIn)", "inputs": ["ORD-1|orderShipsTo|ADDR-9", "ADDR-9|locatedIn|US-WEST"] },
  "support": [ { "ruleId": "chain:orderShipsTo>locatedIn", "pathEdgeIds": ["ORD-1|orderShipsTo|ADDR-9", "ADDR-9|locatedIn|US-WEST"] } ],
  "ts": "2025-11-10T20:22:00Z"
}
```

Deletion and pruning rules - Explicit edges: Re-computed on each write. For each predicate p on the source, any previous explicit dst not present in the new explicit set is deleted (deleteExplicitBySrcNotIn). - Inferred/derived edges: After recomputation, any prior inferred edge for p whose dst is not in the newly inferred set is deleted (deleteInferredBySrcNotIn). - Safety cleanup: A periodic or manual prune removes derived edges that have no support recorded (pruneDerivedWithoutSupport). Use the drift-repair job or a scheduled task to keep storage clean when inputs are removed out-of-band.

Worked examples (source Order = O, Customer = C, Organization = G, Shipment = S, Address = A, Region = R)

1) [placedBy, memberOf] ⇒ placedInOrg

- Inputs: O --placedBy→ C, C --memberOf→ G
- Implied: O --placedInOrg→ G

```
{"src":"O","srcType":"Order","p":"placedInOrg","dst":"G","dstType":"Organization","inferred":true,"derived":true,
  "prov":{"rule":"chain(placedBy,memberOf)","inputs":["O|placedBy|C","C|memberOf|G"]},
  "support":[{"ruleId":"chain:placedBy>memberOf","pathEdgeIds":["O|placedBy|C","C|memberOf|G"]}]}
```

2) [orderHasShipment, shipsTo] ⇒ orderShipsTo

- Inputs: O --orderHasShipment→ S, S --shipsTo→ A

- Implied: $O \text{ --orderShipsTo--} \rightarrow A$

```
{
  "src": "O",
  "srcType": "Order",
  "p": "orderShipsTo",
  "dst": "A",
  "dstType": "Address",
  "inferred": true,
  "derived": true,
  "prov": {
    "rule": "chain(orderHasShipment, shipsTo)",
    "inputs": ["O|orderHasShipment|S", "S|shipsTo|A"]
  },
  "support": [
    {
      "ruleId": "chain:orderHasShipment>shipsTo",
      "pathEdgeIds": ["O|orderHasShipment|S", "S|shipsTo|A"]
    }
  ]
}
```

3) [orderShipsTo, locatedIn] $\Rightarrow$ orderShipsToRegion

- Inputs: $O \text{ --orderShipsTo--} \rightarrow A, A \text{ --locatedIn--} \rightarrow R$
- Implied: $O \text{ --orderShipsToRegion--} \rightarrow R$

```
{
  "src": "O",
  "srcType": "Order",
  "p": "orderShipsToRegion",
  "dst": "R",
  "dstType": "Region",
  "inferred": true,
  "derived": true,
  "prov": {
    "rule": "chain(orderShipsTo, locatedIn)",
    "inputs": ["O|orderShipsTo|A", "A|locatedIn|R"]
  },
  "support": [
    {
      "ruleId": "chain:orderShipsTo>locatedIn",
      "pathEdgeIds": ["O|orderShipsTo|A", "A|locatedIn|R"]
    }
  ]
}
```

4) [placedInOrg, ancestorOf] $\Rightarrow$ placedInOrg

- Inputs: $O \text{ --placedInOrg--} \rightarrow G1, G1 \text{ --ancestorOf--} \rightarrow G2$ (ancestorOf is transitive)
- Implied: $O \text{ --placedInOrg--} \rightarrow G2$ (can repeat over the ancestor chain)

```
{
  "src": "O",
  "srcType": "Order",
  "p": "placedInOrg",
  "dst": "G2",
  "dstType": "Organization",
  "inferred": true,
  "derived": true,
  "prov": {
    "rule": "chain(placedInOrg, ancestorOf)",
    "inputs": ["O|placedInOrg|G1", "G1|ancestorOf|G2"]
  },
  "support": [
    {
      "ruleId": "chain:placedInOrg>ancestorOf",
      "pathEdgeIds": ["O|placedInOrg|G1", "G1|ancestorOf|G2"]
    }
  ]
}
```

Operational notes - If you delete inputs (e.g., remove $C.memberOf=G$), the next write of O (or a reindex) will prune stale placedInOrg edges. If inputs are removed out-of-band, run drift-repair to prune unsupported derived edges. - Always scope operations by realm/tenant. Edges are partitioned by dataDomain.tenantId and indexed accordingly.